



TIME SERIES FORECASTING - ABC ESTATE WINES - ROSE WINE

Project Report

Project Details:

Project Name	Time Series Forecasting - ABC Estate Wines - Rose Wine
Original Author	Reshma B
Date	08-12-2024

Contents

1. BACKGROUND.....	8
2. OBJECTIVE.....	9
3. DATA DESCRIPTION.....	10
4. EXPLORATORY DATA ANALYSIS.....	11
4.1. Data structure.....	11
4.2. Data types.....	11
4.3. Null values	11
4.4. Duplicate Values	12
4.5. Missing Value Treatment.....	12
4.6. Setting index for the data.....	13
4.7. Statistical Summary	13
4.8. Time series data Plot	14
4.9. Boxplot of the variation of Rose wine with respect to months across years	15
4.10. Boxplot of the variation of Rose wine across years	16
4.11. Empirical Cumulative Distribution Function (ECDF) Plot	17
4.12. Graph of Rose wine sales with respect to months across years	17
5. INSIGHTS ON EXPLORATORY DATA ANALYSIS	18
6. TIME SERIES DECOMPOSITION.....	19
6.1. Additive Decomposition.....	19
6.1.1. Additive Decomposition Plot	19
6.1.2. Additive Decomposition Values	20
6.1.3. Additive Decomposition Residual Plot	21
6.2. Multiplicative Decomposition.....	21
6.2.1. Multiplicative Decomposition Plot	22
6.2.2. Multiplicative Decomposition values	23
6.2.3. Multiplicative Decomposition Residual Plot	24
6.3. Insights on time series decomposition	24
7. DATA PRE-PROCESSING	25
7.1. Train-Test Split	25
8. MODEL BUILDING – ORIGINAL DATA	27
8.1. LINEAR REGRESSION.....	27
8.1.1. Model Building	27
8.1.2. Model Evaluation.....	28
8.2. SIMPLE AVERAGE.....	28

8.2.1.	Model Building	28
8.2.2.	Model Evaluation.....	29
8.3.	MOVING AVERAGE	30
8.3.1.	Model Building	30
8.3.2.	Model Evaluation.....	31
8.3.3.	Combined Plots of Linear Regression, Simple Average and Moving Average Models	32
8.4.	SIMPLE EXPONENTIAL SMOOTHING (SES).....	33
8.4.1.	Model Building	33
8.4.2.	Model Evaluation.....	34
8.4.3.	Different values of alpha (0.01-1)	34
8.5.	DOUBLE EXPONENTIAL SMOOTHING (DES)/HOLT'S MODEL	35
8.5.1.	Model Building	36
8.5.2.	Model Evaluation.....	37
8.5.3.	Different values of alpha and beta (0.01-1)	37
8.6.	TRIPLE EXPONENTIAL SMOOTHING (TES)/HOLT-WINTER'S MODEL	38
8.6.1.	Additive Trend and Multiplicative seasonality	39
8.6.1.1.	Model Building	39
8.6.1.2.	Model Evaluation.....	40
8.6.1.3.	Different values of alpha, beta and gamma (0.01-1).....	40
8.6.2.	Additive Trend and Additive seasonality	41
8.6.2.1.	Model Building	41
8.6.2.2.	Model Evaluation.....	42
8.6.2.3.	Different values of alpha, beta and gamma (0.01-1).....	43
8.6.3.	Multiplicative Trend and Multiplicative seasonality	44
8.6.3.1.	Model Building	44
8.6.3.2.	Model Evaluation.....	45
8.6.4.	Multiplicative Trend and Additive seasonality	45
8.6.4.1.	Model Building	45
8.6.4.2.	Model Evaluation.....	46
8.7.	Combined Plots of SES, DES and TES Models	47
9.	CHECK FOR STATIONARITY	48
10.	MODEL BUILDING - STATIONARY DATA	50
10.1.	AutoCorrelation Function (ACF) and Partial AutoCorrelation Function (PACF)	50
10.2.	Manual ARIMA	52
10.2.1.	Model Building	52
10.2.2.	Model Evaluation.....	53

10.3.	Auto ARIMA	54
10.3.1.	Model Building	54
10.3.2.	Model Evaluation.....	56
10.4.	Manual SARIMA	56
10.4.1.	Model Building	56
10.4.2.	Model Evaluation.....	58
10.5.	Auto SARIMA	58
10.5.1.	Model Building	58
10.5.2.	Model Evaluation.....	60
10.6.	Comparison of ARIMA and SARIMA models	61
11.	COMPARISON OF PERFORMANCE OF THE MODELS.....	62
12.	REBUILDING THE BEST MODEL USING THE ENTIRE DATA	63
12.1.	Model Building	63
12.2.	Model Evaluation.....	63
13.	FORECAST FOR THE NEXT 12 MONTHS	64
14.	ACTIONABLE INSIGHTS AND RECOMMENDATIONS	65

LIST OF TABLES:

Table #	Table Title
Table 1	Data structure
Table 2	Data types
Table 3	Datetime index
Table 4	Statistical summary
Table 5	Additive Decomposition: Trend-Seasonality-Residual values
Table 6	Multiplicative Decomposition: Trend-Seasonality-Residual values
Table 7	Train-Test Split
Table 8	Linear Regression Model
Table 9	Mean Forecast Values
Table 10	Rolling mean values
Table 11	SES parameters
Table 12	Alpha and RMSE scores for SES Model
Table 13	DES parameters
Table 14	Alpha, Beta and RMSE scores for DES Model
Table 15	TESam parameters
Table 16	Alpha, Beta, Gamma and RMSE scores for TESam Model
Table 17	TESaa parameters
Table 18	Alpha, Beta, Gamma and RMSE scores for TESaa Model
Table 19	TESmm parameters
Table 20	TESma parameters
Table 21	Manual ARIMA results
Table 22	Auto ARIMA AIC and RMSE values
Table 23	Auto ARIMA results
Table 24	Manual SARIMA results
Table 25	Auto SARIMA AIC and RMSE values
Table 26	Auto SARIMA results
Table 27	Comparison of RMSE values of Models

LIST OF FIGURES:

FIGURE #	FIGURE TITLE
Fig 1	Time Series Plot
Fig 2	Boxplot: Variation of Rose wine with respect to months across years
Fig 3	Boxplot: Variation of Rose wine across years (Outliers)
Fig 4	Boxplot: Variation of Rose wine across years (Without Outliers)
Fig 5	ECDF Plot
Fig 6	Rose wine sales with respect to months across years
Fig 7	Additive Decomposition
Fig 8	Additive Decomposition: Residual Plot
Fig 9	Multiplicative Decomposition
Fig 10	Multiplicative Decomposition: Residual Plot
Fig 11	Train-Test Split
Fig 12	Linear Regression Model on Test
Fig 13	Simple Average Model on Test
Fig 14	Moving Average Model on Whole data
Fig 15	Moving Average Model on Test
Fig 16	Linear Regression, Simple Average and Moving Average Models on Test
Fig 17	SES Model on Test ($\alpha=0.1027$)
Fig 18	SES Model on Test ($\alpha=0.99$)
Fig 19	DES Model on Test ($\alpha=0.1519$ $\beta=0.1519$)
Fig 20	DES Model on Test ($\alpha=0.20$ $\beta=0.05$)
Fig 21	TESam Model on Test ($\alpha=0.0995$ $\beta=0.00015$ $\gamma=0.0018$)
Fig 22	TESam Model on Test ($\alpha=0.1$ $\beta=1.0$ $\gamma=0.2$)
Fig 23	TESaa Model on Test ($\alpha=0.089$ $\beta=1.663$ $\gamma=0.00016$)
Fig 24	TESaa Model on Test ($\alpha=0.1$ $\beta=0.1$ $\gamma=0.5$)
Fig 25	TESmm Model on Test ($\alpha=0.067$ $\beta=5.32$ $\gamma=1.87$)
Fig 26	TESma Model on Test ($\alpha=0.026$ $\beta=1.456$ $\gamma=0.0038$)
Fig 27	SES, DES and TES Models on Test
Fig 28	Dicker-Fuller Test-1
Fig 29	Dicker-Fuller Test-2
Fig 30	Stationary time series
Fig 31	ACF Plot
Fig 32	PACF Plot
Fig 33	Manual ARIMA
Fig 34	Auto ARIMA
Fig 35	Manual SARIMA
Fig 36	Auto SARIMA
Fig 37	Auto SARIMA Diagnostics Plot
Fig 38	Comparison of ARIMA and SARIMA models
Fig 39	TESam model ($\alpha=0.1$ $\beta=1.0$ $\gamma=0.2$) on entire data
Fig 40	Forecast for next 12 months

1. BACKGROUND

ABC Estate Wines is a renowned producer of a wide variety of wines. The historical sales data for the rose wine variety, spanning the 20th century is obtained with the goal of boosting sales performance. By identifying emerging market opportunities and reinforcing our competitive advantage in the wine industry, these insights will inform strategic decision-making, optimize inventory management, enhance marketing efforts, and ultimately improve the company's sales strategies moving forward.

2. OBJECTIVE

Our primary focus is to analyse the data using time series forecasting to:

- To explore the data, identify trends, patterns, and key factors that have influenced wine sales throughout the 20th century.
- By applying data analytics and forecasting methods, to derive actionable insights that will support strategic decision-making and optimize future sales strategies.
- To analyse and predict wine sales trends for the next 12 months using the historical data provided by ABC Estate Wines.
- To empower ABC Estate Wines with valuable insights and foresight, enabling them to improve sales performance, seize emerging market opportunities, and sustain a competitive advantage in the wine industry.

3. DATA DESCRIPTION

The dataset includes:

- A time series spanning from 1980 to 1985, represented in the '**YearMonth**' column.
- The number of units sold for rose wine, recorded in the '**Rose**' column.

4. EXPLORATORY DATA ANALYSIS

4.1. Data structure

After importing NumPy and Pandas for tasks like data reading, data computation and data manipulation and loading data, the following observations regarding the data structure were noted.

data.head()		
	YearMonth	Rose
0	1980-01	112.0
1	1980-02	118.0
2	1980-03	129.0
3	1980-04	99.0
4	1980-05	116.0

Table 1. Data structure

OBSERVATION	VARIABLES
187	2

4.2. Data types

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 187 entries, 0 to 186
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   YearMonth   187 non-null    object
1   Rose        185 non-null    float64
dtypes: float64(1), object(1)
memory usage: 3.1+ KB
```

Table 2. Data types

- The data set contains two variables indexed from 0-1.
- The data set consists of 1 numerical column, '**Rose**' of float data type.
- The '**YearMonth**' column is in the object data type which has to be converted into date format and further needs to be indexed for performing forecasting analysis.

4.3. Null values

- Two null values were found in the data set at index 174 and 175.

	YearMonth	Rose
172	1994-05	44.0
173	1994-06	45.0
174	1994-07	NaN
175	1994-08	NaN
176	1994-09	46.0
177	1994-10	51.0
178	1994-11	63.0
179	1994-12	84.0
180	1995-01	30.0
181	1995-02	39.0
182	1995-03	45.0
183	1995-04	52.0
184	1995-05	28.0
185	1995-06	40.0
186	1995-07	62.0

4.4. Duplicate Values

- No duplicate values were found in the data set.

4.5. Missing Value Treatment

- The missing values are imputed using an interpolation method called **linear interpolation**, where the missing values are estimated by drawing straight lines between known values.

	YearMonth	Rose
172	1994-05	44
173	1994-06	45
174	1994-07	45
175	1994-08	46
176	1994-09	46
177	1994-10	51
178	1994-11	63
179	1994-12	84
180	1995-01	30
181	1995-02	39
182	1995-03	45
183	1995-04	52
184	1995-05	28
185	1995-06	40
186	1995-07	62

4.6. Setting index for the data

- The **'YearMonth'** column is converted into datetime objects for easy manipulation and time series analysis.
- This makes it easier to work with time series data since the index will represent the temporal aspect of the data.

DatetimeIndex(['1980-01-01', '1980-02-01', '1980-03-01', '1980-04-01', '1980-05-01', '1980-06-01', '1980-07-01', '1980-08-01', '1980-09-01', '1980-10-01', ... '1994-10-01', '1994-11-01', '1994-12-01', '1995-01-01', '1995-02-01', '1995-03-01', '1995-04-01', '1995-05-01', '1995-06-01', '1995-07-01'], dtype='datetime64[ns]', name='YearMonth', length=187, freq=None)	
	Rose
YearMonth	
1980-01-01	112
1980-02-01	118
1980-03-01	129
1980-04-01	99
1980-05-01	116

Table 3. Datetime index

4.7. Statistical Summary

	Rose
count	187.000000
mean	89.914439
std	39.238356
min	28.000000
25%	62.500000
50%	85.000000
75%	111.000000
max	267.000000

Table 4. Statistical summary

The following observations were made:

- The total number of time series data points is **187**.
- The average monthly sales of rose wine are **90** units.
- The minimum sales recorded in a month are **28** units, while the maximum sales in a month are **267** units.
- 75% of the time, the wine sales are found to be **111** units.

4.8. Time series data Plot

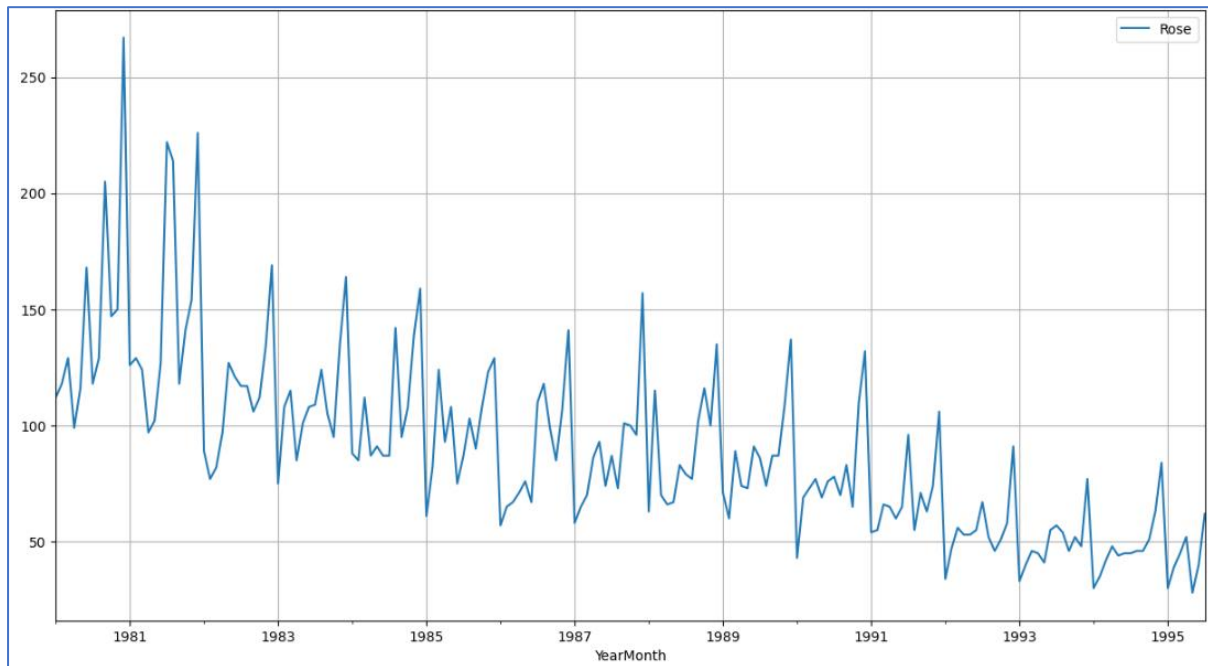


Fig 1. Time Series Plot

OBSERVATIONS:

- The data reveals a **downward trend** in sales from 1980 to 1995, with the highest sales recorded in 1981.
- The data likely represents **multiplicative seasonality** because the fluctuations seem to decline in proportion to the decreasing trend.

4.9. Boxplot of the variation of Rose wine with respect to months across years

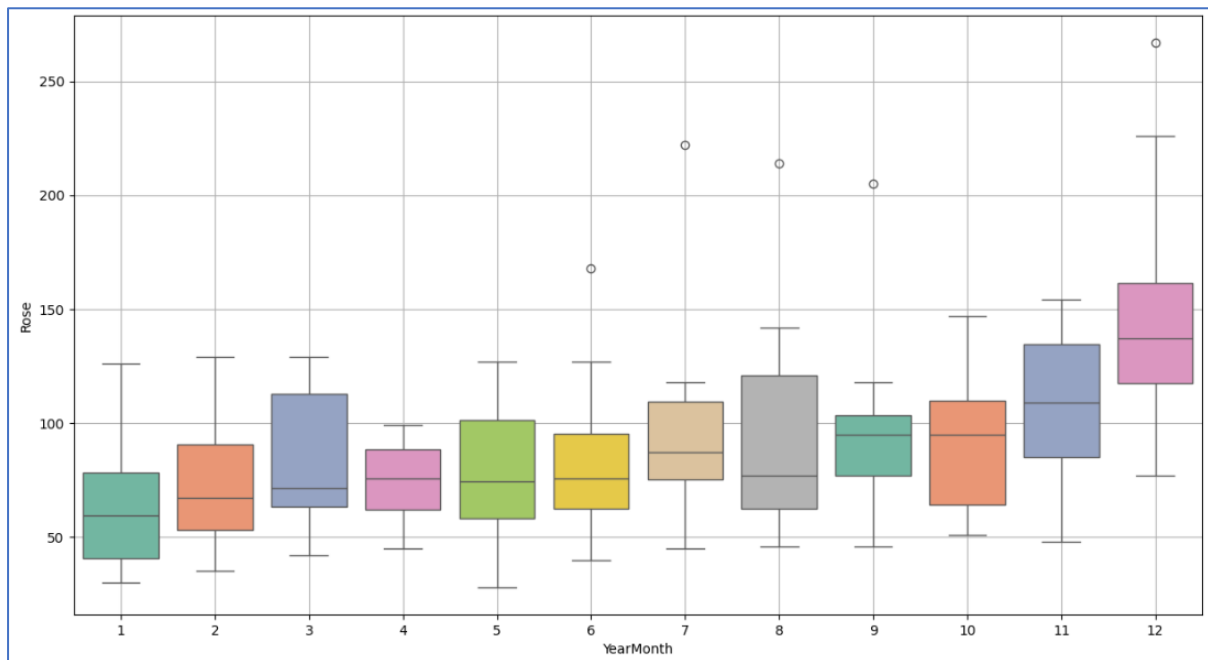


Fig 2. Boxplot: Variation of Rose wine with respect to months across years

OBSERVATIONS:

- **December** consistently shows the highest sales figures for rose wine with an average of 140 units being sold. The pattern suggests that there is **seasonality** or events that drive higher demand for rose wine during the holiday season, such as festivities, parties, and celebrations typically associated with that time of year.
- After December, **November** ranks second in terms of sales, with an average of 120 units sold per month. **October** ranks third in sales, though its average is likely to be slightly lower than November's sales.
- The **trend** suggests that the demand for rose wine begins to rise as the year approaches its end, with November and October experiencing higher sales likely due to upcoming holidays, promotions, and events leading up to December.

4.10. Boxplot of the variation of Rose wine across years

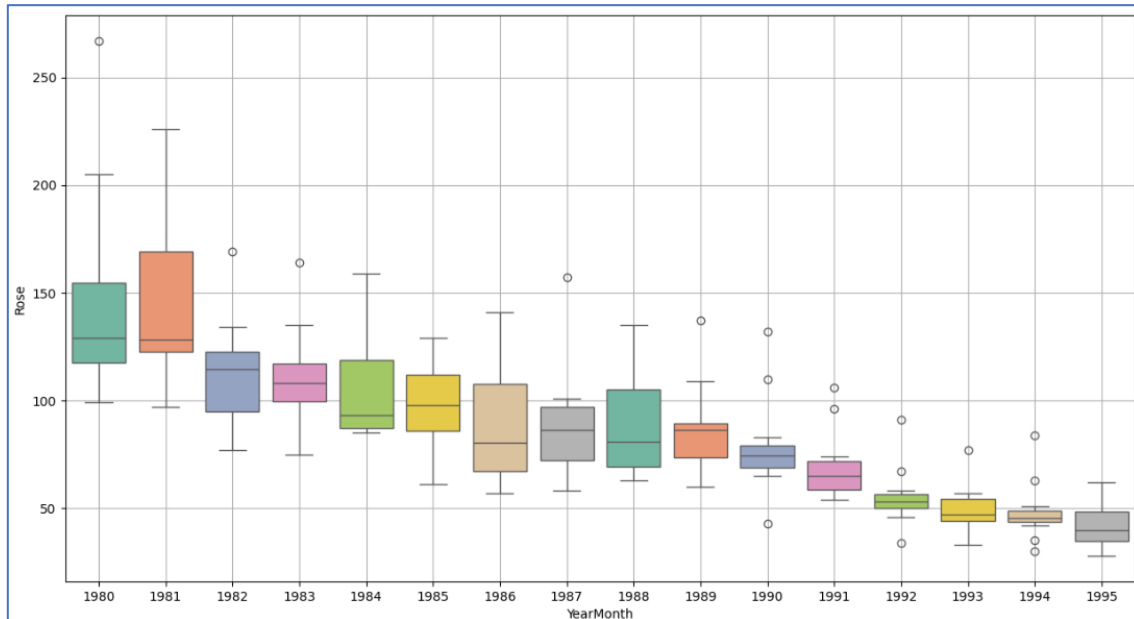


Fig 3. Boxplot: Variation of Rose wine across years (Outliers)

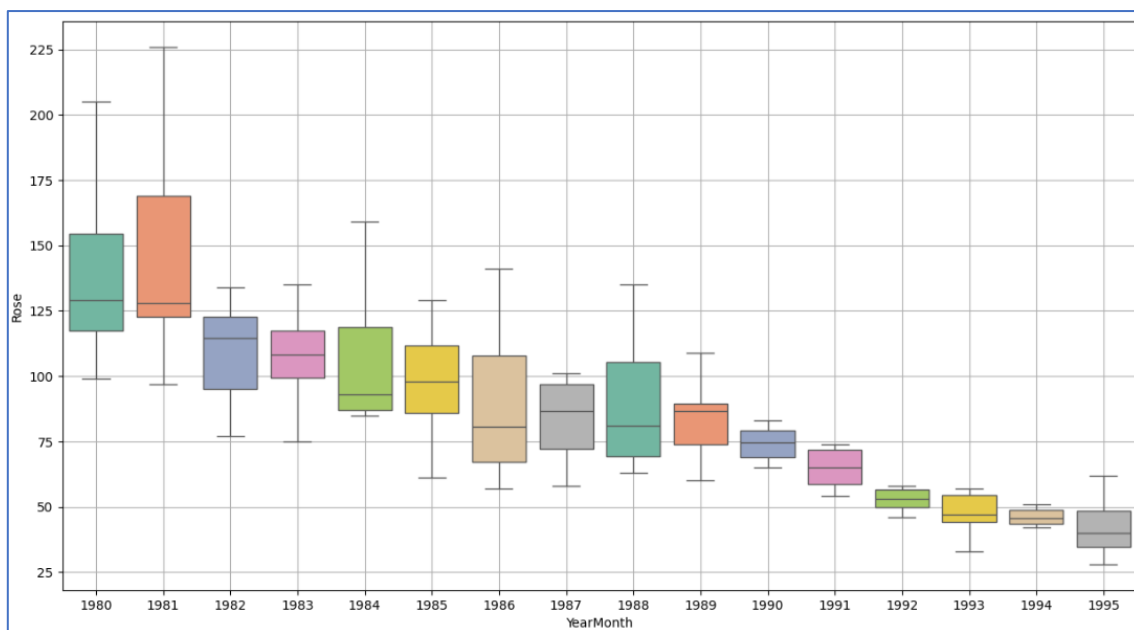


Fig 4. Boxplot: Variation of Rose wine across years (Without Outliers)

OBSERVATIONS:

- From 1980 to 1995, the sales of rose wine show a gradual decrease. This indicates that the sales of rose wine is declining with the increase in year.
- **1980** is identified as the year with the **highest wine sales** with average sales of 125 units followed by **1981**. This suggests that there was a particularly strong market demand for rose wine during that year. This could have been influenced by special events, an increase in popularity, or a successful marketing push.
- After 1982, there is a noticeable decline in sales throughout the years.

4.11. Empirical Cumulative Distribution Function (ECDF) Plot

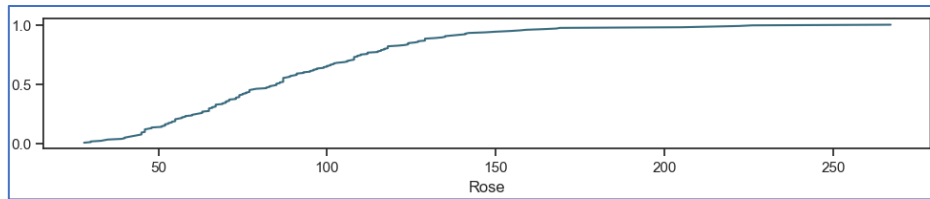


Fig 5. ECDF Plot

OBSERVATIONS:

- **90%** of the total sales values fall below or are equal to **150** units. The majority of sales are concentrated around 150 units, with a significant portion of the data points falling in this range.
- The high concentration of sales around **150** units suggests that the company is consistently achieving sales in this range for most of the time periods. It may indicate that 150 units represents an expected sales figure for most of the observed periods.
- More than **50%** of the data points are greater than or equal to **75** units. This suggests that the company regularly sells more than 75 units in half of the time periods indicating a relatively high baseline level of sales across the data.

4.12. Graph of Rose wine sales with respect to months across years

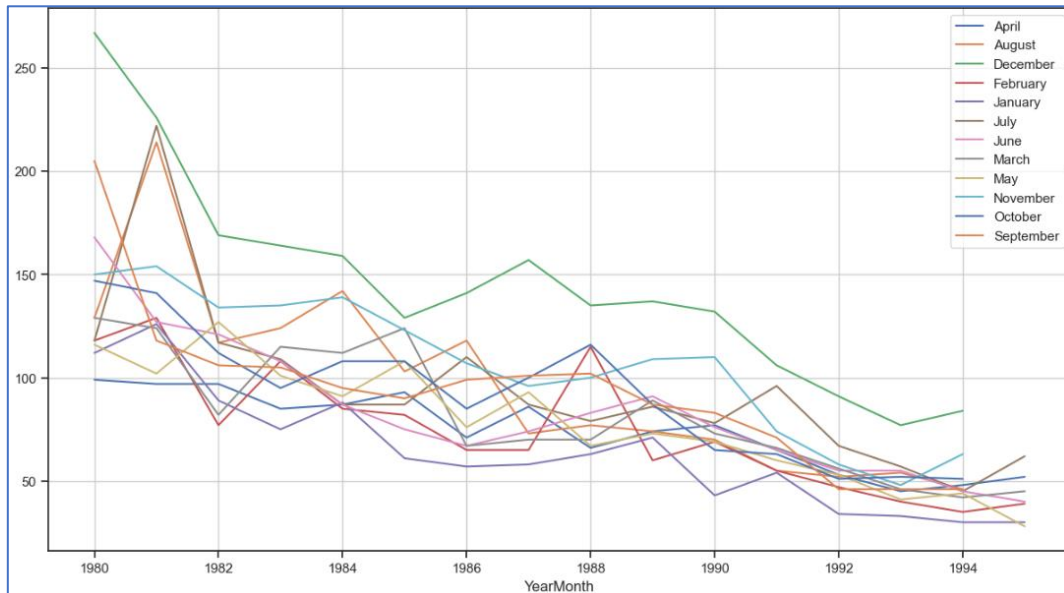


Fig 6. Rose wine sales with respect to months across years

OBSERVATIONS:

- **December** consistently shows the **highest sales** across all years, a peak that no other month ever reaches. This period can be identified as the **prime wine sales season**.
- **November** also experiences notable sales peaks with occasional fluctuations.
- **January** and **February** are all other months have **lowest sales**, indicating a seasonal dip in wine sales, likely due to the festive offseason.

5. INSIGHTS ON EXPLORATORY DATA ANALYSIS

- The time series data has both **trend** and **seasonality**.
- **December** experiences the **highest sales** of rose wine, closely followed by November. The spike in sales during these months suggests that they are naturally recognized as prime wine sales periods, requiring little to no marketing or advertising efforts.
- Conversely, the other months need targeted campaigns and strategies to boost sales.
- Special attention should be given to **January** and **February** to drive sales during these quieter months.
- Wine sales in the remaining months experience a significant drop, requiring interventions to prevent further decline.
- A gradual decline in sales is observed, necessitating the implementation of strategies to revive and sustain sales in the coming years.

6. TIME SERIES DECOMPOSITION

Time Series Decomposition is a technique used to break down a time series dataset into its fundamental components: **Trend, Seasonality and Residual (or noise)**. This helps to better understand the underlying patterns in the data, and can be very useful for forecasting and anomaly detection.

6.1. Additive Decomposition

In an additive decomposition, the trend, seasonality and residual are added together to give the observed data. This is typically used when the amplitude of the seasonal fluctuations remains constant throughout the dataset, i.e., seasonality has a constant effect regardless of the trend.

6.1.1. Additive Decomposition Plot

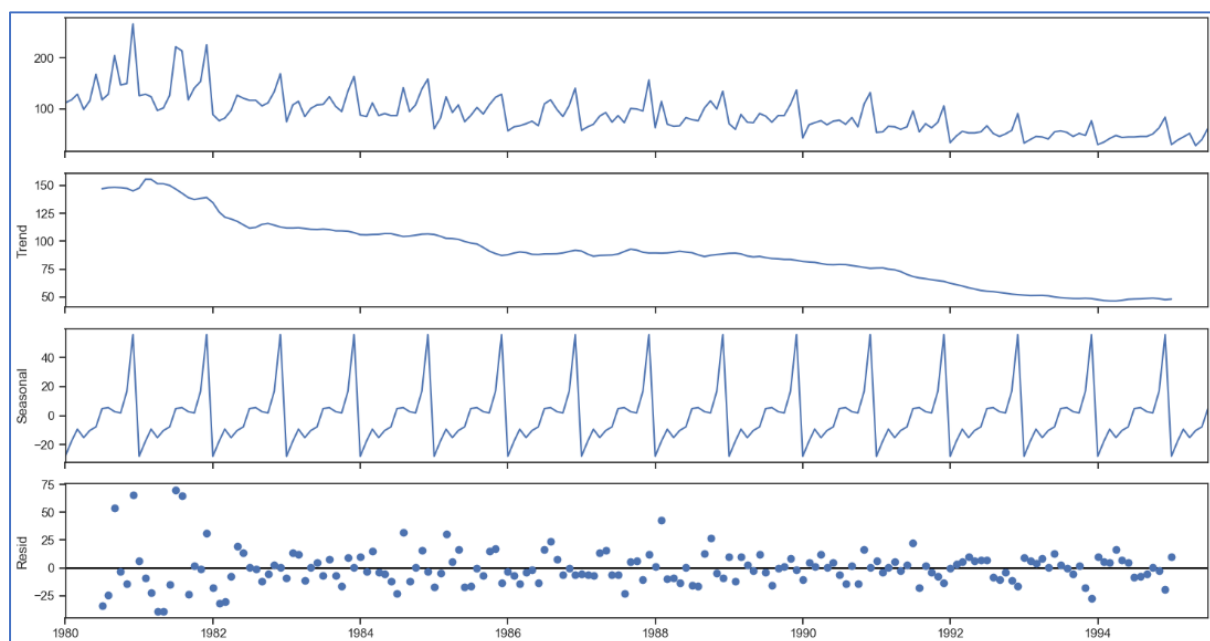


Fig 7. Additive Decomposition

6.1.2. Additive Decomposition Values

```

Trend
YearMonth
1980-01-01      NaN
1980-02-01      NaN
1980-03-01      NaN
1980-04-01      NaN
1980-05-01      NaN
1980-06-01      NaN
1980-07-01    147.083333
1980-08-01    148.125000
1980-09-01    148.375000
1980-10-01    148.083333
1980-11-01    147.416667
1980-12-01    145.125000
Name: trend, dtype: float64

Seasonality
YearMonth
1980-01-01   -27.908730
1980-02-01   -17.434722
1980-03-01    -9.285913
1980-04-01   -15.098413
1980-05-01   -10.196627
1980-06-01    -7.678770
1980-07-01    4.874603
1980-08-01    5.521825
1980-09-01    2.774603
1980-10-01    1.871825
1980-11-01   16.846825
1980-12-01   55.713492
Name: seasonal, dtype: float64

Residual
YearMonth
1980-01-01      NaN
1980-02-01      NaN
1980-03-01      NaN
1980-04-01      NaN
1980-05-01      NaN
1980-06-01      NaN
1980-07-01   -33.957937
1980-08-01   -24.646825
1980-09-01   53.850397
1980-10-01   -2.955159
1980-11-01   -14.263492
1980-12-01   66.161508
Name: resid, dtype: float64

```

Table 5. Additive Decomposition: Trend-Seasonality-Residual values

6.1.3. Additive Decomposition Residual Plot

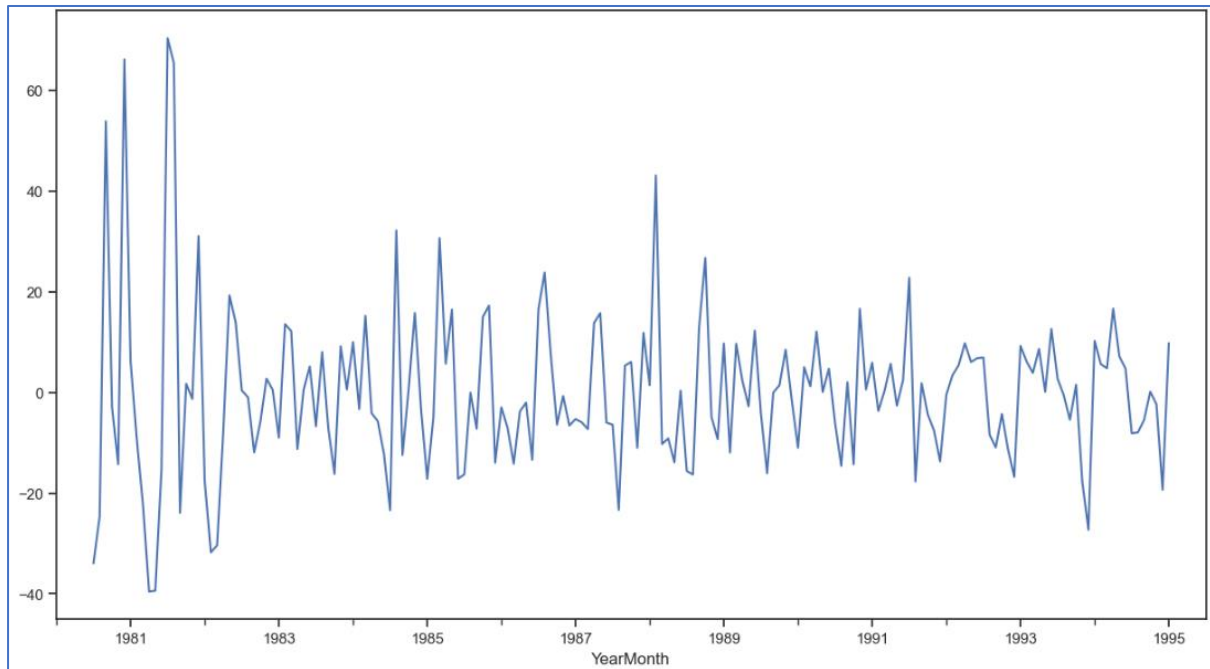


Fig 8. Additive Decomposition: Residual Plot

OBSERVATIONS:

- The trend, seasonality, and residuals have been effectively separated.
- A downward trend is observed, which experiences a gradual decline.
- The seasonality is clearly visible, with distinct patterns occurring each year.
- The residuals are centred around zero, displaying randomness.

6.2. Multiplicative Decomposition

In a multiplicative decomposition, the trend, seasonality, and residuals are multiplied together to produce the observed data. This approach is typically used when the seasonal fluctuations are proportional to the level of the trend, that is, the size of seasonal fluctuations increases or decreases in magnitude as the overall trend increases or decreases.

6.2.1. Multiplicative Decomposition Plot

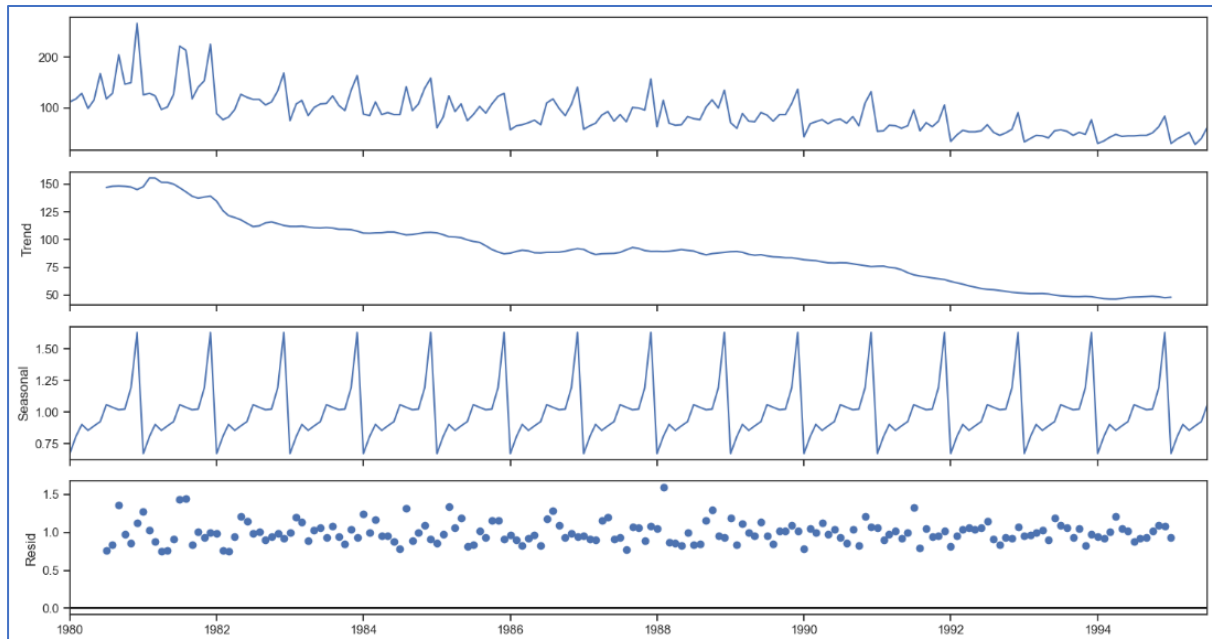


Fig 9. Multiplicative Decomposition

6.2.2. Multiplicative Decomposition values

```

Trend
YearMonth
1980-01-01      NaN
1980-02-01      NaN
1980-03-01      NaN
1980-04-01      NaN
1980-05-01      NaN
1980-06-01      NaN
1980-07-01    147.083333
1980-08-01    148.125000
1980-09-01    148.375000
1980-10-01    148.083333
1980-11-01    147.416667
1980-12-01    145.125000
Name: trend, dtype: float64

Seasonality
YearMonth
1980-01-01    0.670111
1980-02-01    0.806178
1980-03-01    0.901163
1980-04-01    0.854023
1980-05-01    0.889414
1980-06-01    0.923984
1980-07-01    1.057576
1980-08-01    1.036339
1980-09-01    1.017647
1980-10-01    1.022572
1980-11-01    1.192347
1980-12-01    1.628644
Name: seasonal, dtype: float64

Residual
YearMonth
1980-01-01      NaN
1980-02-01      NaN
1980-03-01      NaN
1980-04-01      NaN
1980-05-01      NaN
1980-06-01      NaN
1980-07-01    0.758590
1980-08-01    0.840349
1980-09-01    1.357675
1980-10-01    0.970772
1980-11-01    0.853379
1980-12-01    1.129647
Name: resid, dtype: float64

```

Table 6. Multiplicative Decomposition: Trend-Seasonality-Residual values

6.2.3. Multiplicative Decomposition Residual Plot

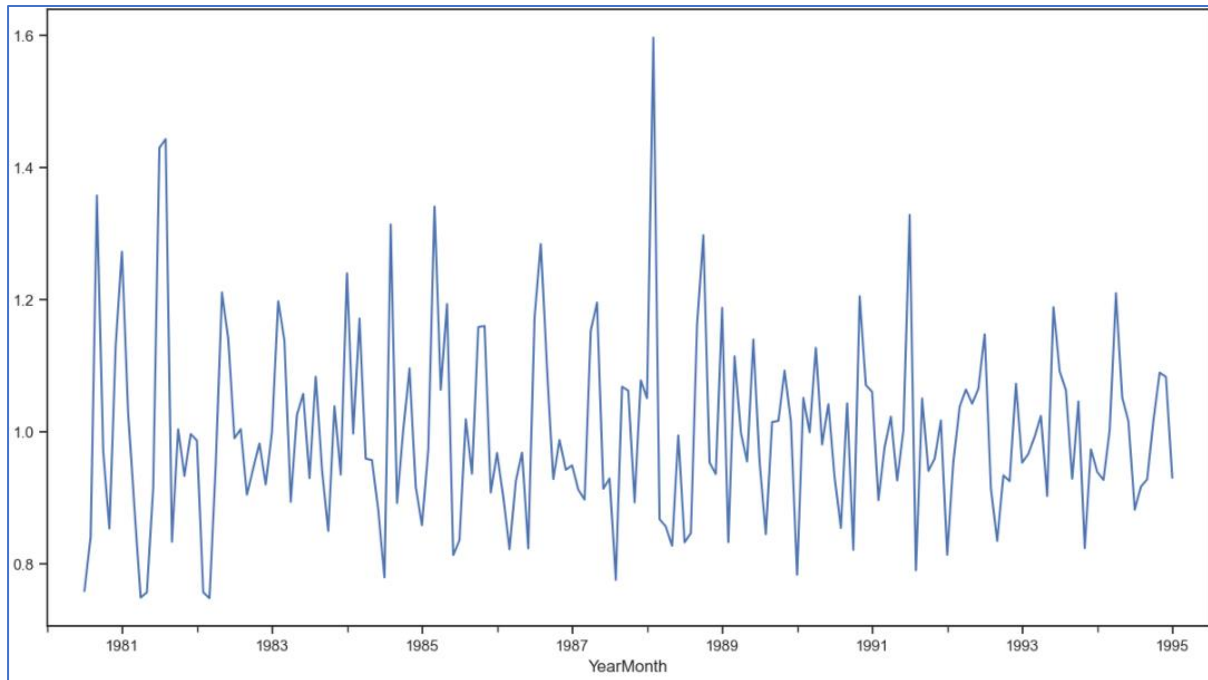


Fig 10. Multiplicative Decomposition: Residual Plot

OBSERVATIONS:

- The trend, seasonality, and residuals have been effectively separated.
- A downward trend is observed, which gradually declines.
- The seasonality is clearly visible, with distinct patterns occurring each year.
- The residuals are centred around 1, displaying randomness.

6.3. Insights on time series decomposition

- The multiplicative model shows more **precise** and **proportionate** residuals (clustered around 1), it might be a better fit for the data, especially if the data has significant percentage-based seasonal effects.
- However, the additive model's residuals are less concentrated around 0, this indicates a potential mismatch between the model and the data's structure.

7. DATA PRE-PROCESSING

7.1. Train-Test Split

- The data set has been split into training and test set with a split of **70%** and **30%** respectively.
- The train set has **130** observations and the test set has **57** observations.

```
print(train.shape)
print(test.shape)
```

```
(130, 1)
(57, 1)
```

First few rows of Training Data

Rose	
YearMonth	
1980-01-01	112
1980-02-01	118
1980-03-01	129
1980-04-01	99
1980-05-01	116

Last few rows of Training Data

Rose	
YearMonth	
1990-06-01	76
1990-07-01	78
1990-08-01	70
1990-09-01	83
1990-10-01	65

First few rows of Test Data

Rose	
YearMonth	
1990-11-01	110
1990-12-01	132
1991-01-01	54
1991-02-01	55
1991-03-01	66

Last few rows of Test Data

Rose	
YearMonth	
1995-03-01	45
1995-04-01	52
1995-05-01	28
1995-06-01	40
1995-07-01	62

Table 7. Train-Test Split

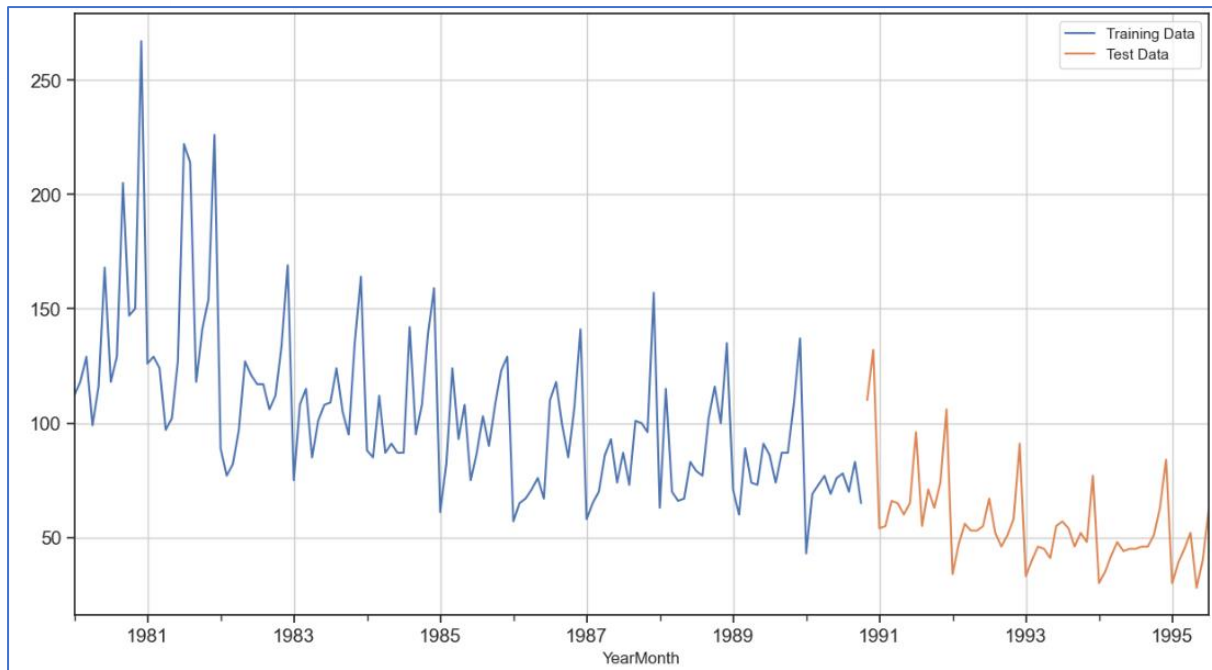


Fig 11. Train-Test Split

8. MODEL BUILDING – ORIGINAL DATA

8.1. LINEAR REGRESSION

In time series forecasting, linear regression can be used to model the relationship between the target variable, here wine sales and time. This is often done when the data exhibits trends that can be approximated by a linear relationship.

8.1.1. Model Building

The data is split into train and test sets and the linear regression model is trained on the train data and forecasted on the test data.

```
from sklearn.linear_model import LinearRegression
Model_1 = LinearRegression()
Model_1.fit(LinearRegression_train[['time']],LinearRegression_train['Sparkling'])
```

LinearRegression

LinearRegression()

Table 8. Linear Regression Model

The below plot gives the linear regression model performance.

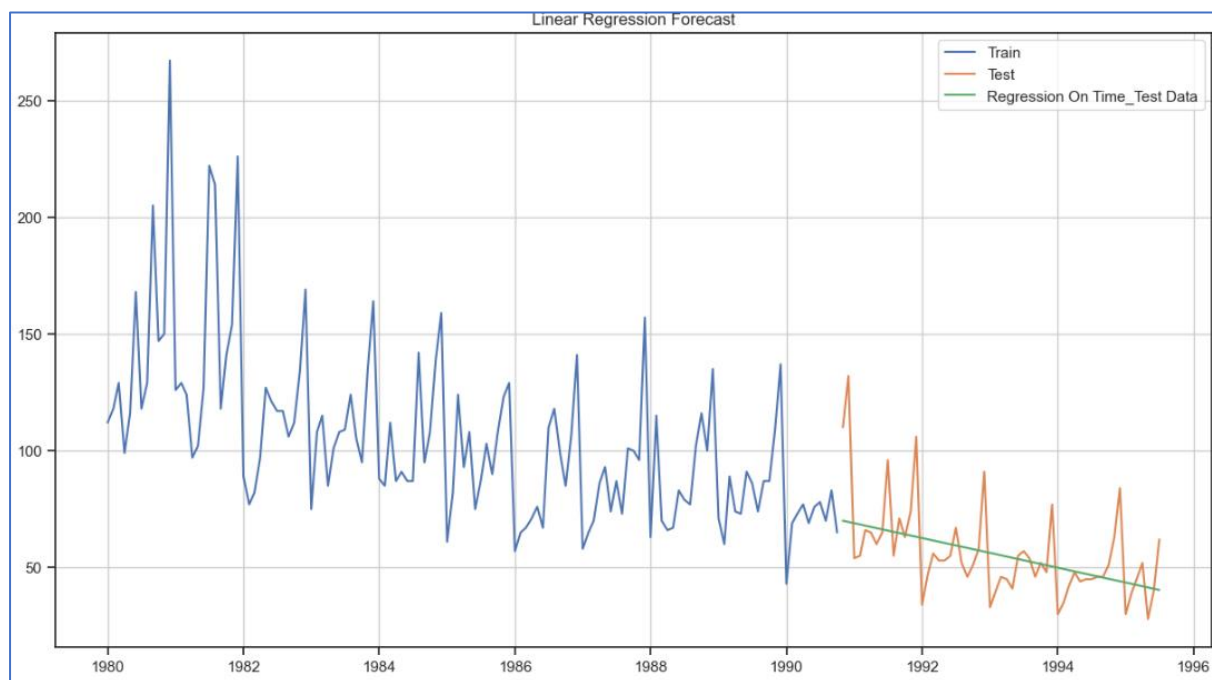


Fig 12. Linear Regression Model on Test

8.1.2. Model Evaluation

The Root Mean Squared Error (RMSE) is a widely used metric for evaluating the performance of a regression model, especially in the context of time series forecasting. It measures the average magnitude of the error between predicted values and actual values, providing a way to quantify how well the model is performing.

The RMSE of the model is calculated.

```
from sklearn import metrics
Model_1_Test_RMSE = metrics.mean_squared_error(test['Rose'],Model_1_Test_Predict,squared=False)
print("For RegressionOnTime forecast on the Test Data, RMSE is %3.3f " %(Model_1_Test_RMSE))

For RegressionOnTime forecast on the Test Data, RMSE is 17.356
```

The RMSE value for the Linear Regression Model is **17.356**.

OBSERVATIONS:

- The RMSE value of **17.356** is quite low, and may indicate that the Linear Regression model is one of the models with good fit for the data.
- Linear regression assumes a linear relationship and shows a steady decline without seasonal effects.

8.2. SIMPLE AVERAGE

The Simple Average Model is one of the simplest approaches to forecasting time series data. It predicts future values based on the mean of past observations. This model is typically used as a baseline to compare more sophisticated forecasting methods.

8.2.1. Model Building

The mean forecast values for the train set are calculated and forecasted on the test set.

	Rose	mean_forecast
YearMonth		
1990-11-01	110	104.692308
1990-12-01	132	104.692308
1991-01-01	54	104.692308
1991-02-01	55	104.692308
1991-03-01	66	104.692308

Table 9. Mean Forecast Values

The below plot gives the **simple average model** performance.

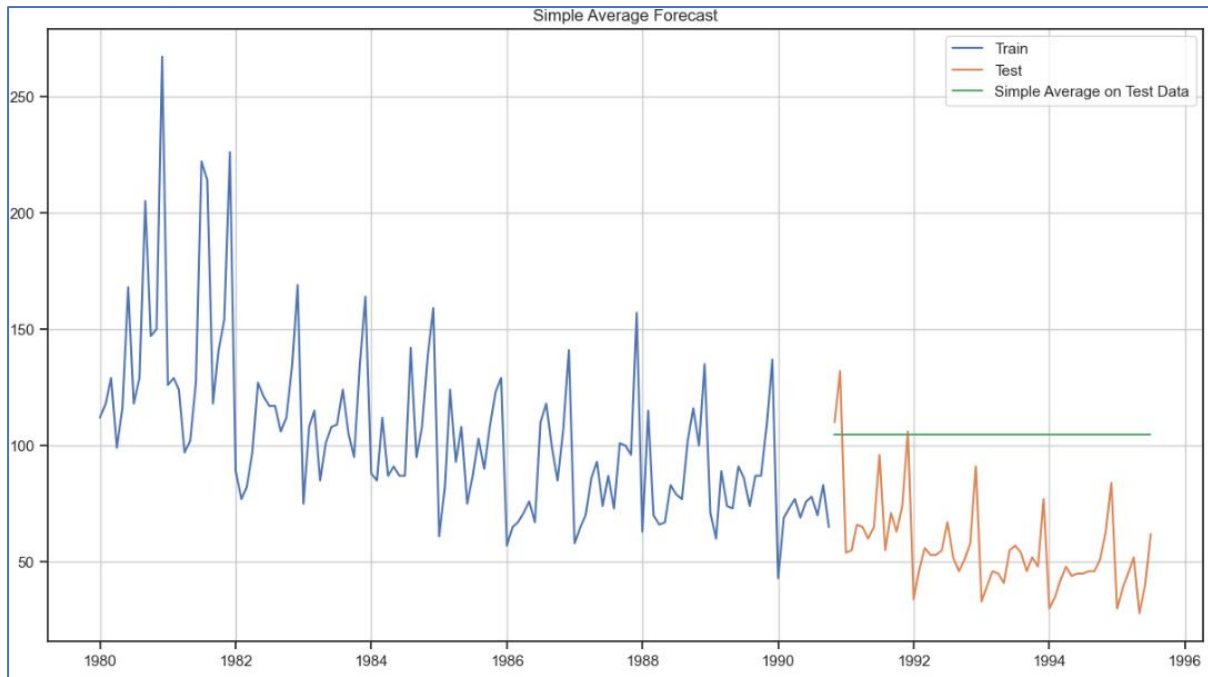


Fig 13. Simple Average Model on Test

8.2.2. Model Evaluation

The RMSE of the model is calculated.

```
Model_2_Test_RMSE = metrics.mean_squared_error(test['Rose'], SimpleAverage_test['mean_forecast'], squared=False)
print("For Simple Average forecast on the Test Data, RMSE is %3.3f" %(Model_2_Test_RMSE))
```

For Simple Average forecast on the Test Data, RMSE is 52.412

The RMSE value for the Simple Average Model is 52.412.

OBSERVATIONS:

- The RMSE for the Simple Average Model is **higher** with a value of **52.412**.
- The model assumes that the future value will always be the same as the average of past observations, regardless of any trends, seasonality, or other patterns in the data. Hence it is not a good fit for the data.

8.3. MOVING AVERAGE

The Moving Average (MA) model is a commonly used time series forecasting technique, especially when the data exhibits noise or randomness and lacks obvious trends or seasonality. The Moving Average model smooths out short-term fluctuations and helps identify longer-term trends.

8.3.1. Model Building

The MA model for the train set is calculated and forecasted on the whole data. The `rolling()` is applied to a moving window over a sequence of data to compute mean over that window. The rolling means for values 2,4,6 and 9 is computed.

	Rose	Trailing_2	Trailing_4	Trailing_6	Trailing_9
YearMonth					
1980-01-01	112	NaN	NaN	NaN	NaN
1980-02-01	118	115.0	NaN	NaN	NaN
1980-03-01	129	123.5	NaN	NaN	NaN
1980-04-01	99	114.0	114.5	NaN	NaN
1980-05-01	116	107.5	115.5	NaN	NaN

Table 10. Rolling mean values

The model is forecasted on the whole data and below plot is obtained.

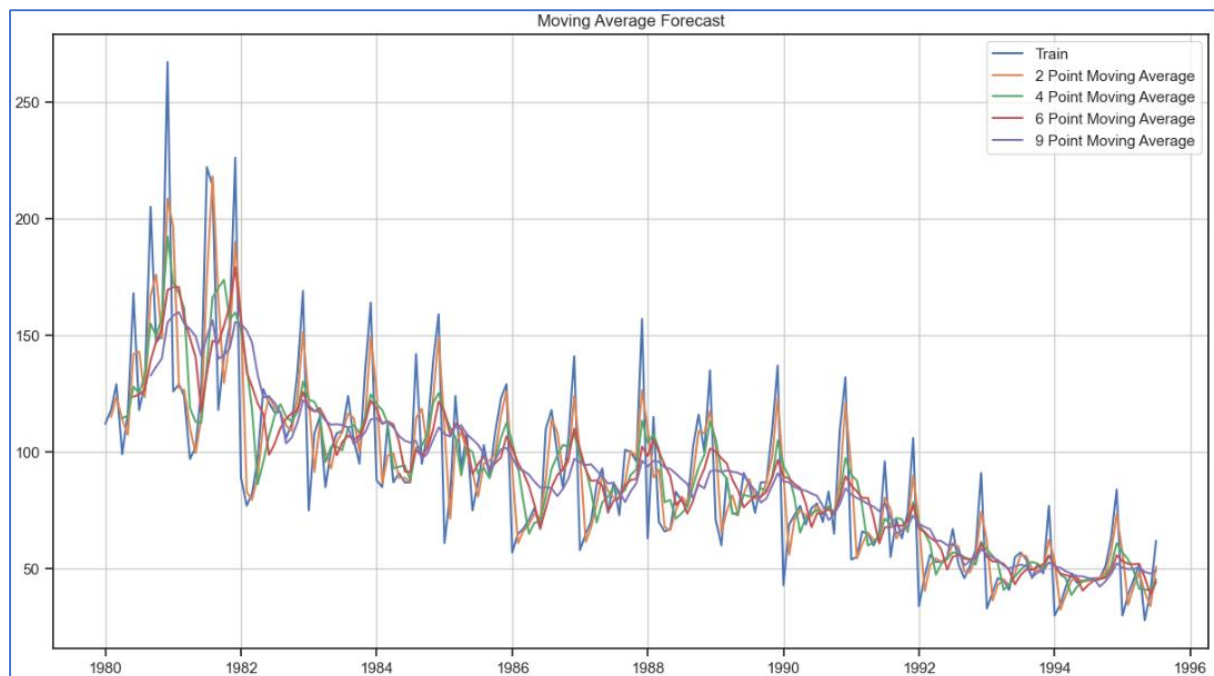


Fig 14. Moving Average Model on Whole data

The MA model is trained on test data and forecasted on the test set to obtain the below plot.

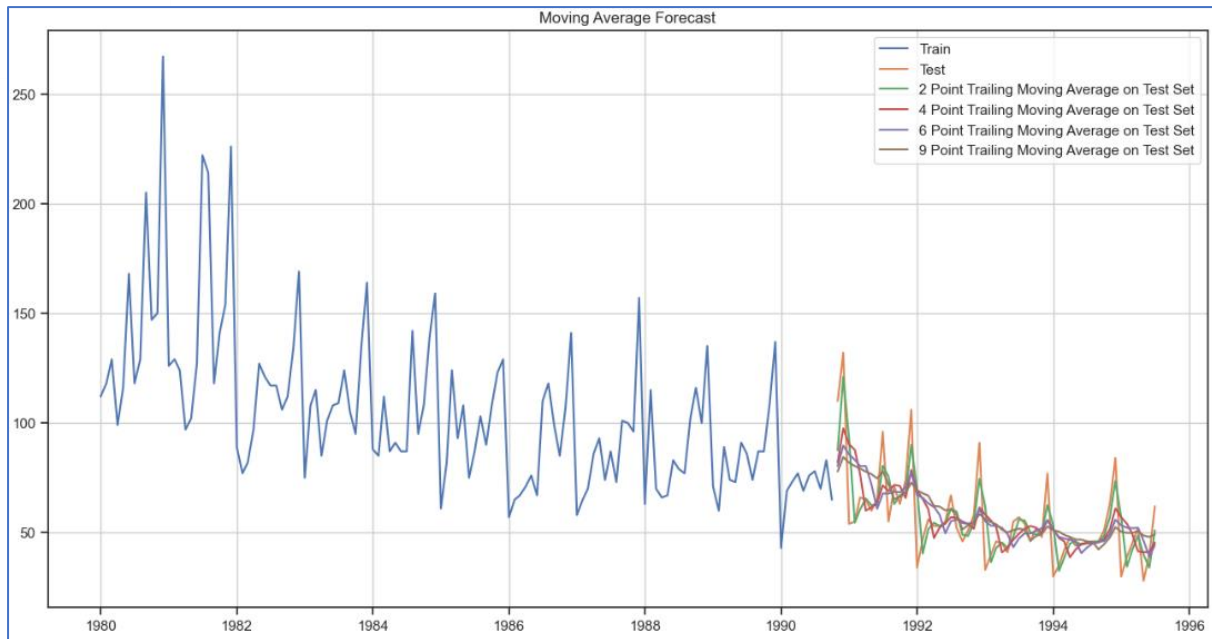


Fig 15. Moving Average Model on Test

8.3.2. Model Evaluation

The RMSE of the model is calculated.

For 2 point Moving Average Model forecast on the Training Data, RMSE is 11.801
 For 4 point Moving Average Model forecast on the Training Data, RMSE is 15.367
 For 6 point Moving Average Model forecast on the Training Data, RMSE is 15.863
 For 9 point Moving Average Model forecast on the Training Data, RMSE is 16.342

The RMSE value for the Moving Average Model with Rolling Mean=2 is the lowest with a value of **11.801**.

OBSERVATIONS:

- The Rolling Mean with a window size of 2 has the lowest RMSE value of **11.801** and is considered a better fit for the model.
- This suggests that the smaller window is effectively capturing the underlying pattern in the time series data while minimizing forecasting error.
- It responds more quickly to short-term changes or fluctuations in the time series, which might make it more adaptable to recent trends or sudden shifts in the data.
- With a **window size of 2**, the model is averaging the last two data points to forecast the next one. This makes it less smoothed compared to larger window sizes (like 4, 6, or 9), and thus it may be more sensitive to rapid changes in the time series.

8.3.3. Combined Plots of Linear Regression, Simple Average and Moving Average Models

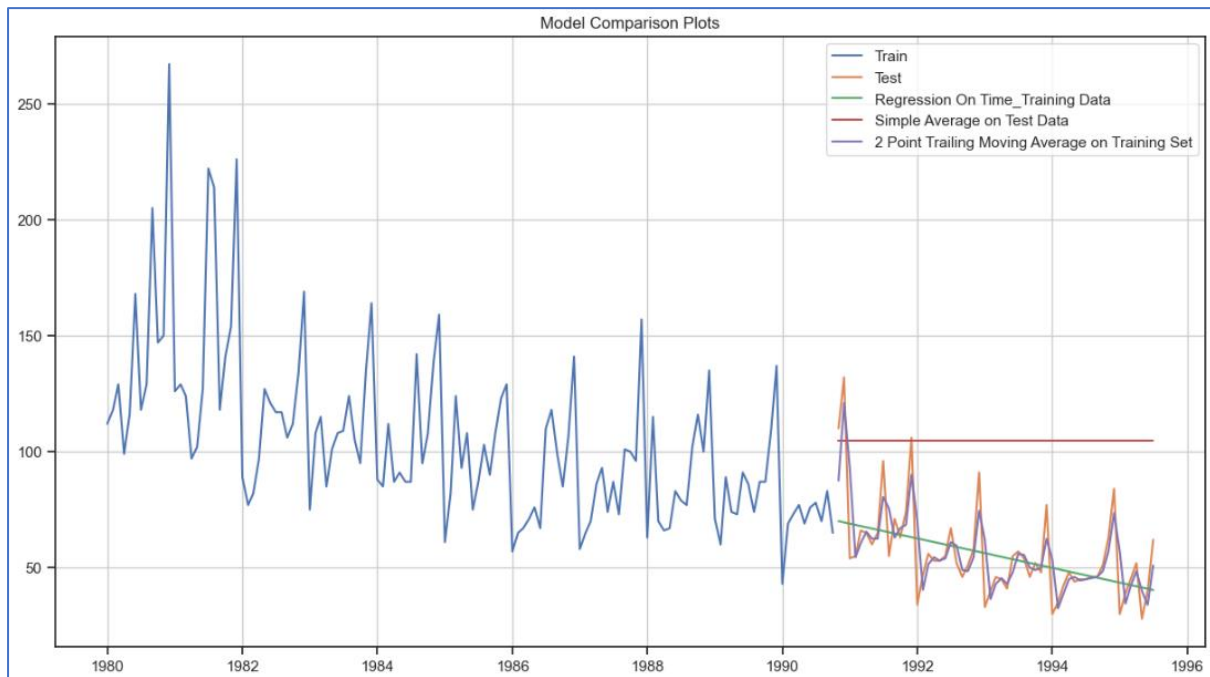


Fig 16. Linear Regression, Simple Average and Moving Average Models on Test

OBSERVATIONS:

- The moving average model with a rolling mean at window 2 is performing well with an RMSE score of **11.801** compared to linear regression and simple average models.

	Test RMSE
Model_1: RegressionOnTime	17.356199
Model_2: SimpleAverageModel	52.412167
Model_3: 2pointTrailingMovingAverage	11.801167
Model_3: 4pointTrailingMovingAverage	15.366538
Model_3: 6pointTrailingMovingAverage	15.863483
Model_3: 9pointTrailingMovingAverage	16.342156

- This window size is very small, so the model is very responsive to recent data and captures short-term fluctuations effectively.
- This suggests that the MA model with a smaller window is effectively capturing the dynamics of the time series and providing more accurate predictions.
- This window size is very small, so the model is very responsive to recent data and captures short-term fluctuations effectively.

8.4. SIMPLE EXPONENTIAL SMOOTHING (SES)

Simple Exponential Smoothing (SES) is a forecasting method used for time series data. It is most effective when the data does not exhibit any clear trend or seasonality. The primary idea behind SES is to smooth the time series data by weighting past observations and giving more weight to the recent observations.

The SES comprises a **smoothing level, alpha** that takes a value between 0 and 1 which controls how much weight is given to the most recent observation. The value of alpha determines how quickly the weights decay for past observations.

8.4.1. Model Building

The SES model is trained on the train set and forecasted on the test set. The below are the parameters.

```
model_SES_autofit.params
{'smoothing_level': 0.10272100493683696,
'smoothing_trend': nan,
'smoothing_seasonal': nan,
'damping_trend': nan,
'initial_level': 134.26284082377956,
'initial_trend': nan,
'initial_seasons': array([], dtype=float64),
'use_boxcox': False,
'lamda': None,
'remove_bias': False}
```

Table 11. SES parameters

The SES model with a smoothing level, **alpha=0.1027** gives the below performance on the test.

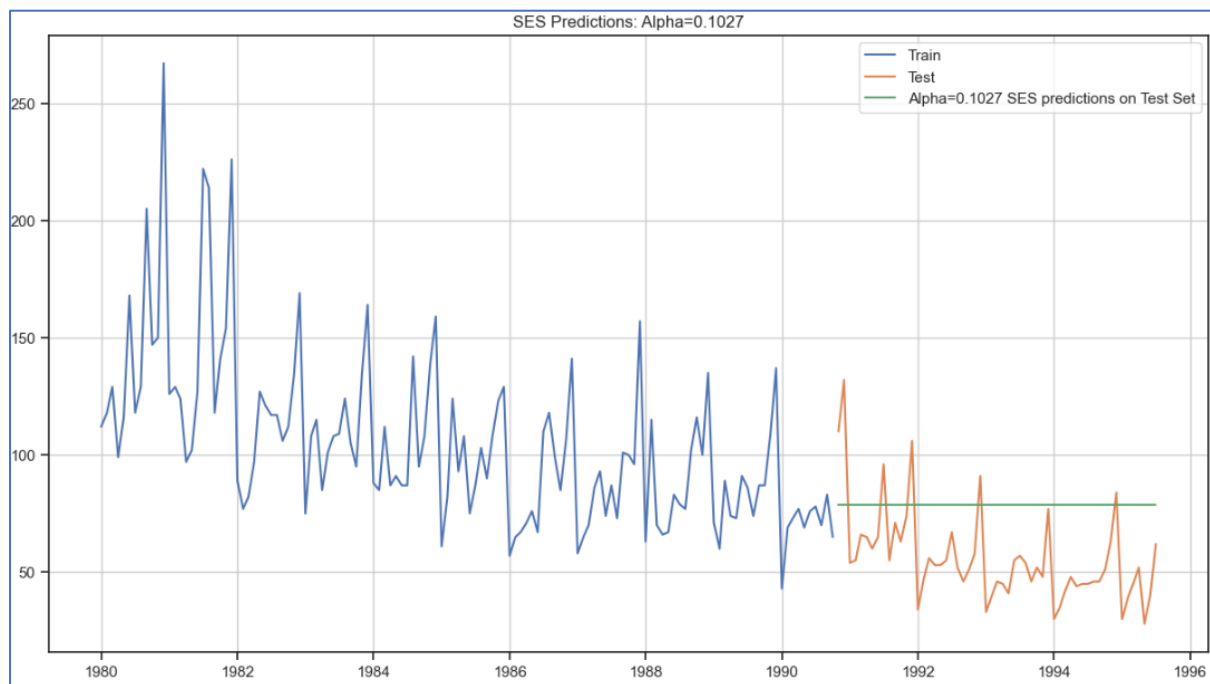


Fig 17. SES Model on Test (alpha=0.1027)

8.4.2. Model Evaluation

The RMSE of the model is calculated.

```
print('SES RMSE:',mean_squared_error(test.values,SES_predict.values,squared=False))
SES RMSE: 30.18845492817289
```

The RMSE value for the SES Model (alpha=0.1027) is 30.188.

OBSERVATIONS:

- The SES model with alpha=0.1027 results in a high RMSE of 30.188, which suggests that this value of alpha is not suitable for the time series data.
- The low alpha value means that the model is giving very little weight to recent data and instead relying heavily on previous forecasts. It assumes that the data is highly stable and that recent changes or fluctuations are not important.
- Thereby SES with alpha=0.1027 is not a better fit for the data.

8.4.3. Different values of alpha (0.01-1)

The different values of alpha ranging from 0.01-1 that yield lower RMSE scores are obtained.

	Alpha Values	Test RMSE
98	0.99	21.839882
97	0.98	21.912806
96	0.97	21.985838
95	0.96	22.058937
94	0.95	22.132066
...	...	...
4	0.05	33.912661
3	0.04	35.447602
2	0.03	37.809389
1	0.02	41.514110
0	0.01	46.487836

Table 12. Alpha and RMSE scores for SES Model

From the above table, **alpha=0.99** is found to be a better value with the lowest RMSE score. Hence the SES model is forecasted on the test data to see the performance of the model at the lowest RMSE score. The below plot is obtained.

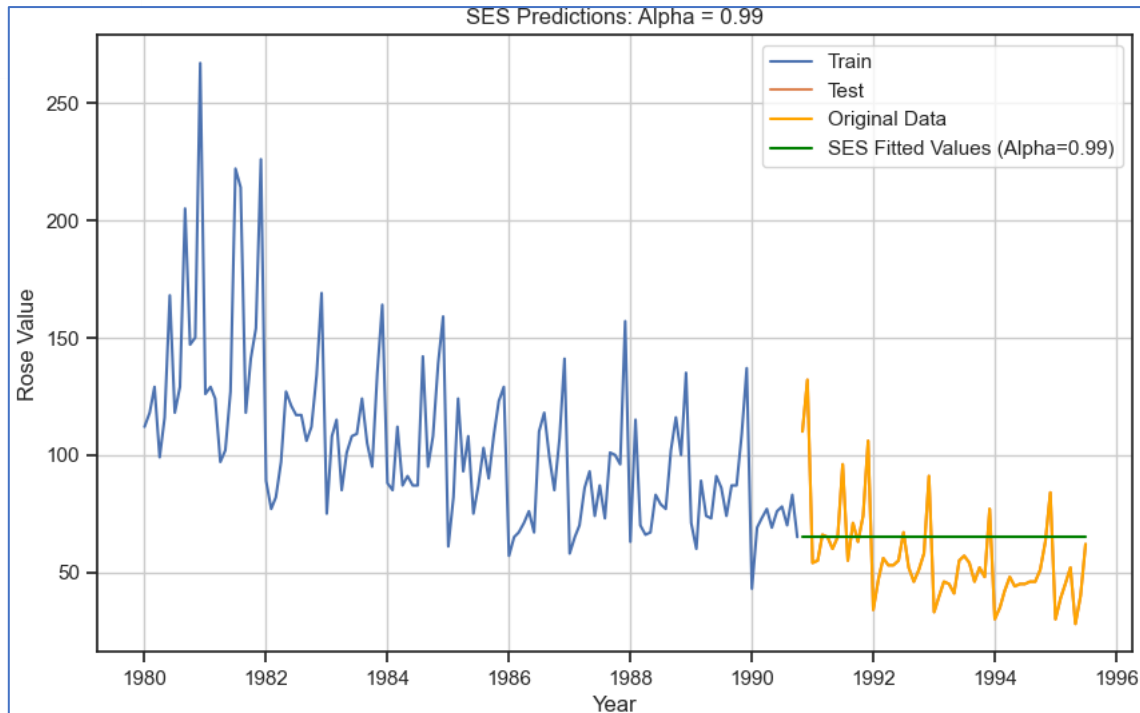


Fig 18. SES Model on Test (alpha=0.99)

OBSERVATIONS:

- The SES model with **alpha=0.99** still results in a high RMSE of **21.83**.
- The alpha=0.99 outperforms alpha=0.1027 to a slighter level.
- However, from the plot, the SES model is not a good fit for the time series data.

8.5. DOUBLE EXPONENTIAL SMOOTHING (DES)/HOLT'S MODEL

Double Exponential Smoothing (DES) is an extension of SES that is used when the time series data exhibits a trend (but no seasonality). While SES only models the level of the data, DES adds a second equation to capture the trend component of the series. This method is effective for time series with non-stationary data, where there is a consistent upward or downward trend.

The DES comprises a **smoothing level, alpha** which controls the weight given to the most recent observed value in the level equation and **smoothing trend, beta** which controls the weight given to the most recent trend estimate.

8.5.1. Model Building

The DES model is trained on the train set and forecasted on the test set. The below are the parameters.

model_DES_autofit.params_formatted			
	name	param	optimized
smoothing_level	alpha	0.151942	True
smoothing_trend	beta	0.151942	True
initial_level	l.0	112.000000	False
initial_trend	b.0	6.000000	False

Table 13. DES parameters

The DES model with **alpha=0.1519** and **beta=0.1519** gives the below performance on the test.

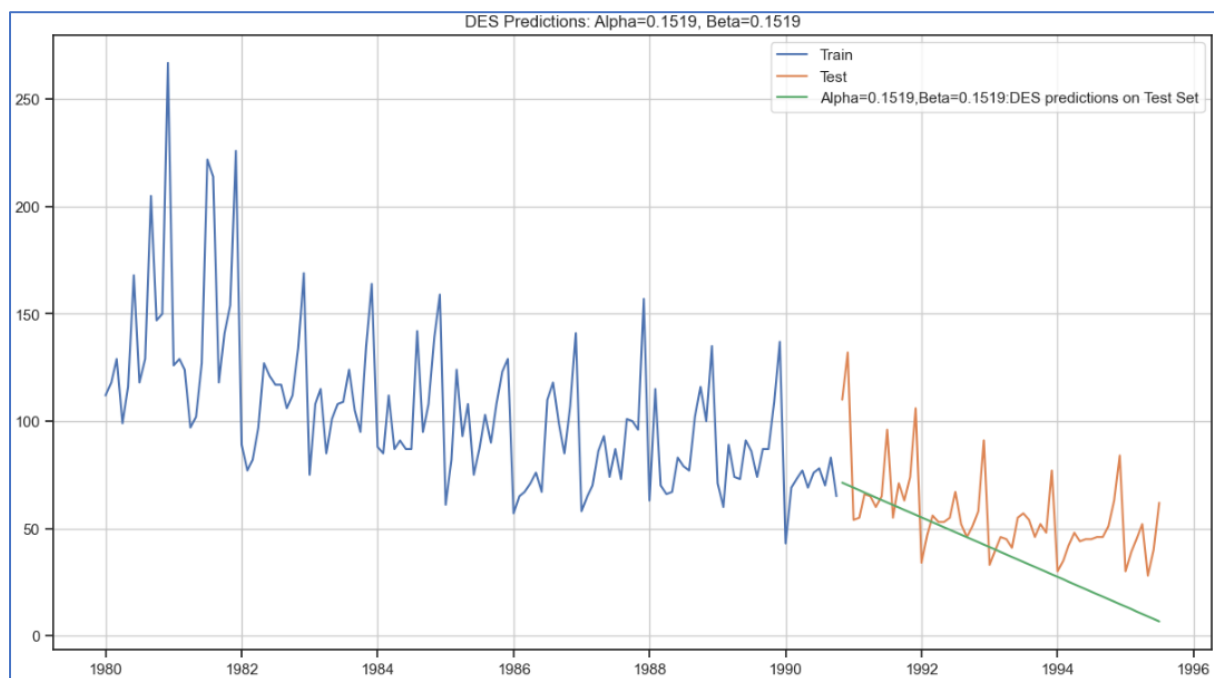


Fig 19. DES Model on Test (alpha=0.1519 beta=0.1519)

8.5.2. Model Evaluation

The RMSE of the model is calculated.

```
rmse_model3_test = metrics.mean_squared_error(test['Rose'],DES_predict,squared=False)
print("For DES forecast on the Test Data, RMSE is %3.3f" %(rmse_model3_test))
For DES forecast on the Test Data, RMSE is 26.050
```

The RMSE value for the DES Model (alpha=0.1519 beta=0.1519) is 26.05.

OBSERVATIONS:

- The DES model with **alpha=0.1519** and **beta=0.1519** results in a RMSE value of **26.05**, which is the highest and the seasonality is completely ignored in the DES model.
- This indicates that the model is not a good fit for the time series data.
- Since seasonality is not incorporated, the forecast is not aligning with the actual seasonal variations in the data, contributing to the poor model performance.

8.5.3. Different values of alpha and beta (0.01-1)

The different values of alpha and beta ranging from 0.01-1 that yield lower RMSE scores are obtained.

	Alpha Values	Beta Values	Test RMSE
1885	0.20	0.05	17.287718
4600	0.47	0.47	17.289780
4507	0.46	0.53	17.290610
1490	0.16	0.06	17.291051
1786	0.19	0.05	17.292796
...	...	...	...
9700	0.98	0.98	534.727091
9798	0.99	0.97	541.269487
9701	0.98	0.99	544.114843
9799	0.99	0.98	550.891103
9800	0.99	0.99	560.643212

Table 14. Alpha, Beta and RMSE scores for DES Model

From the above table, **alpha=0.20** and **beta=0.05** is found to be a better value with the lowest RMSE score. Hence the DES model is forecasted on the test data to see the performance of the model at the lowest RMSE score. The below plot is obtained.

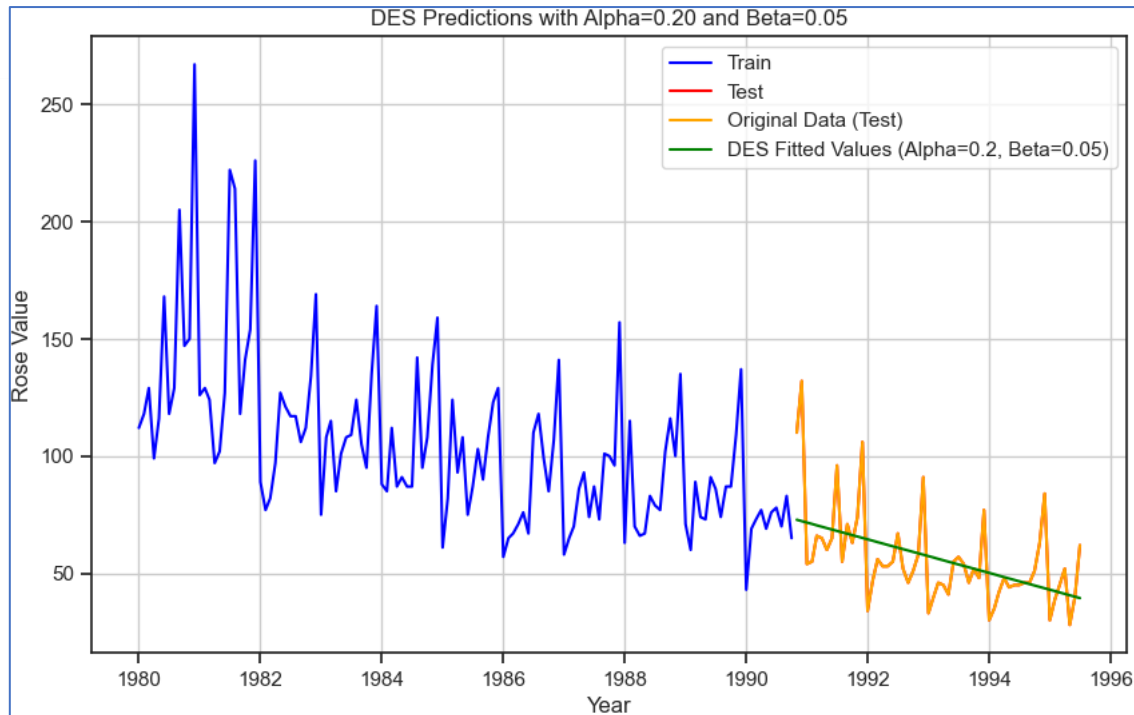


Fig 20. DES Model on Test (alpha=0.20 beta=0.05)

OBSERVATIONS:

- The DES model with **alpha=0.20 and beta=0.05** is almost similar to the linear regression model with a RMSE of **17.28**, but however the model fails to capture the seasonality.
- This suggests that the DES model is not a good fit for the time series data.

8.6. TRIPLE EXPONENTIAL SMOOTHING (TES)/HOLT-WINTER'S MODEL

Triple Exponential Smoothing (TES), also known as Holt-Winters Exponential Smoothing, is an extension of Double Exponential Smoothing (DES) that accounts for seasonality in time series data, in addition to the trend and level components.

The TES model comprises **smoothing level, alpha** for the level component, **smoothing trend, beta** for the trend component and **smoothing seasonal, gamma** for the seasonal component.

8.6.1. Additive Trend and Multiplicative seasonality

8.6.1.1. Model Building

The TES (additive trend and multiplicative seasonality) model, also called **TESam Model** is trained on the train set and forecasted on the test set. The below are the parameters.

```
model_TES_am_autofit.params
{
  'smoothing_level': 0.0995488248939194,
  'smoothing_trend': 0.00015997166258421684,
  'smoothing_seasonal': 0.001852211308409554,
  'damping_trend': nan,
  'initial_level': 170.81628528349015,
  'initial_trend': -0.6912490249257442,
  'initial_seasons': array([0.64171488, 0.72508104, 0.79274121, 0.69502832, 0.78029889,
    0.84087778, 0.925788 , 0.99010997, 0.92666285, 0.91146587,
    1.04793035, 1.44554664]),
  'use_boxcox': False,
  'lamda': None,
  'remove_bias': False}

```

Table 15. TESam parameters

The TESam model with **alpha=0.0995**, **beta=0.00015** and **gamma=0.0018** gives the below performance on the test.

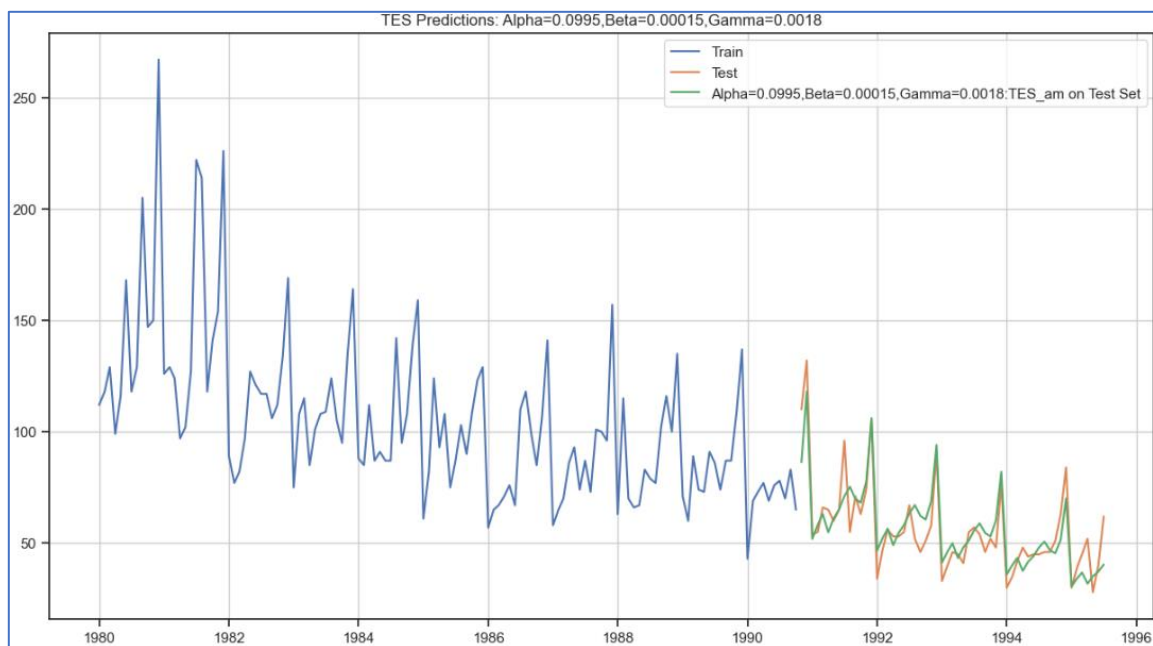


Fig 21. TESam Model on Test (alpha=0.0995 beta=0.00015 gamma=0.0018)

8.6.1.2. Model Evaluation

The RMSE of the model is calculated.

```
modelTES_am= metrics.mean_squared_error(test['Rose'],TES_am_predict,squared=False)
print("For TESam forecast on the Test Data, RMSE is %3.3f" %(modelTES_am))
For TESam forecast on the Test Data, RMSE is 9.323
```

The RMSE value for the TESam Model (alpha=0.0995 beta=0.00015 gamma=0.0018) is 9.323.

OBSERVATIONS:

- The low RMSE value of **9.323** along with the effective capture of level, trend, and seasonality suggests that the TESam model is a good fit for the time series data.
- The model with **alpha=0.0995**, **beta=0.00015** and **gamma=0.0018** is accurately forecasting future values based on past patterns, making it reliable for short-term predictions.
- The plot further confirms this by showing that the model fits the test data well, indicating that it has learned the underlying patterns in the historical data.
- The combination of low error, accurate predictions, and a good visual fit all points to the TES model being an effective and reliable forecasting tool for the data.

8.6.1.3. Different values of alpha, beta and gamma (0.01-1)

The different values of alpha, beta and gamma ranging from 0.01-1 that yield lower RMSE scores are obtained.

	Alpha Values	Beta Values	Gamma Values	Test RMSE
91	0.1	1.0	0.2	9.130274
4	0.1	0.1	0.5	9.216094
5	0.1	0.1	0.6	9.285776
3	0.1	0.1	0.4	9.291278
106	0.2	0.1	0.7	9.335596
...	...	...	...	...
839	0.9	0.4	1.0	325300.586747
488	0.5	0.9	0.9	327275.368108
195	0.2	1.0	0.6	380649.279997
982	1.0	0.9	0.3	478966.918616
587	0.6	0.9	0.8	630045.990584

Table 16. Alpha, Beta, Gamma and RMSE scores for TESam Model

From the above table, **alpha=0.1**, **beta=1.0** and **gamma=0.2** is found to be a better value with the lowest RMSE score. Hence the TESam model is forecasted on the test data to see the performance of the model at the lowest RMSE score. The below plot is obtained.

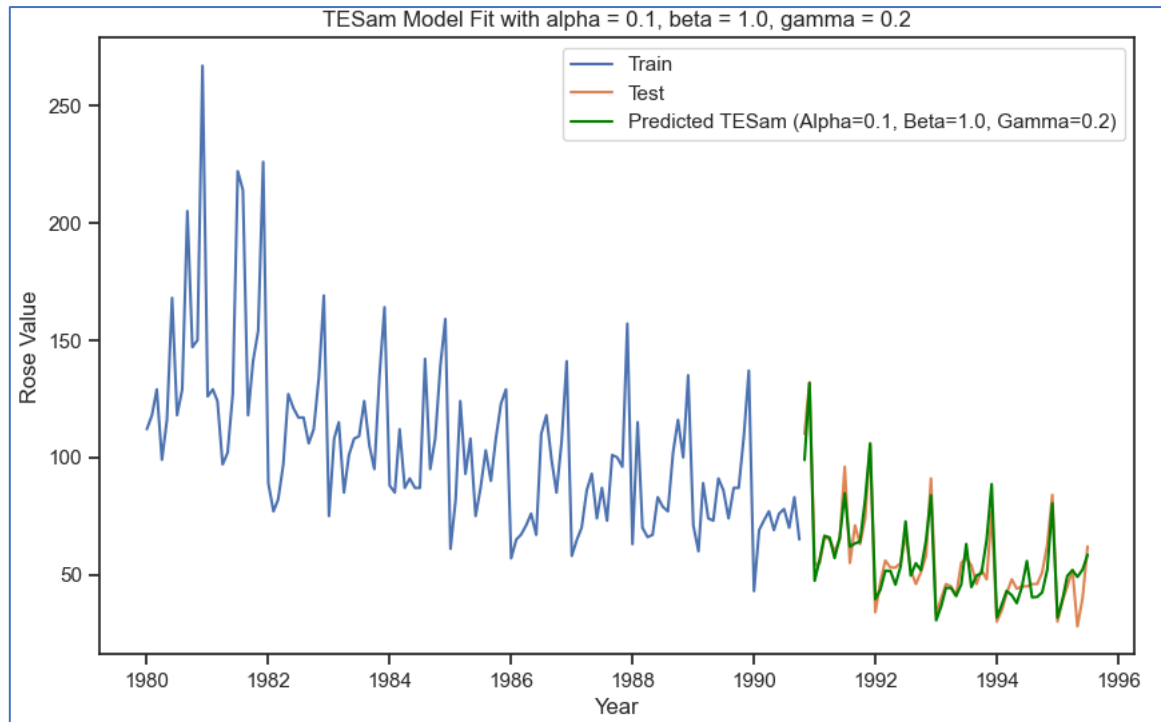


Fig 22. TESam Model on Test (alpha=0.1 beta=1.0 gamma=0.2)

OBSERVATIONS:

- The TESam model with **alpha=0.1**, **beta=1.0** and **gamma=0.2** results in a low RMSE of **9.13** which is relatively lower than RMSE of with **alpha=0.0995**, **beta=0.00015** and **gamma=0.0018**.
- The plot further confirms this by showing that the model fits the test data well, indicating that it has learned the underlying patterns in the historical data.

8.6.2. Additive Trend and Additive seasonality

8.6.2.1. Model Building

The TES (additive trend and additive seasonality) model, also called **TESaa Model** is trained on the train set and forecasted on the test set. The below are the parameters.

```
model_TES_aa_autofit.params
{'smoothing_level': 0.08905538659008487,
'smoothing_trend': 1.6633250764820864e-06,
'smoothing_seasonal': 0.00016051769331753576,
'damping_trend': nan,
'initial_level': 146.8946132087885,
'initial_trend': -0.5564434649146358,
'initial_seasons': array([-31.0791286 , -18.7475215 , -10.84903361, -21.31947355,
-12.57332333, -7.10747397,  2.632544 ,  8.9706052 ,
  4.93120786,  3.14301903,  19.65389231,  63.91326473]),
'use_boxcox': False,
'lamda': None,
'remove_bias': False}
```

Table 17. TESaa parameters

The TESaa model with **alpha=0.089**, **beta=1.663** and **gamma=0.00016** gives the below performance on the test.

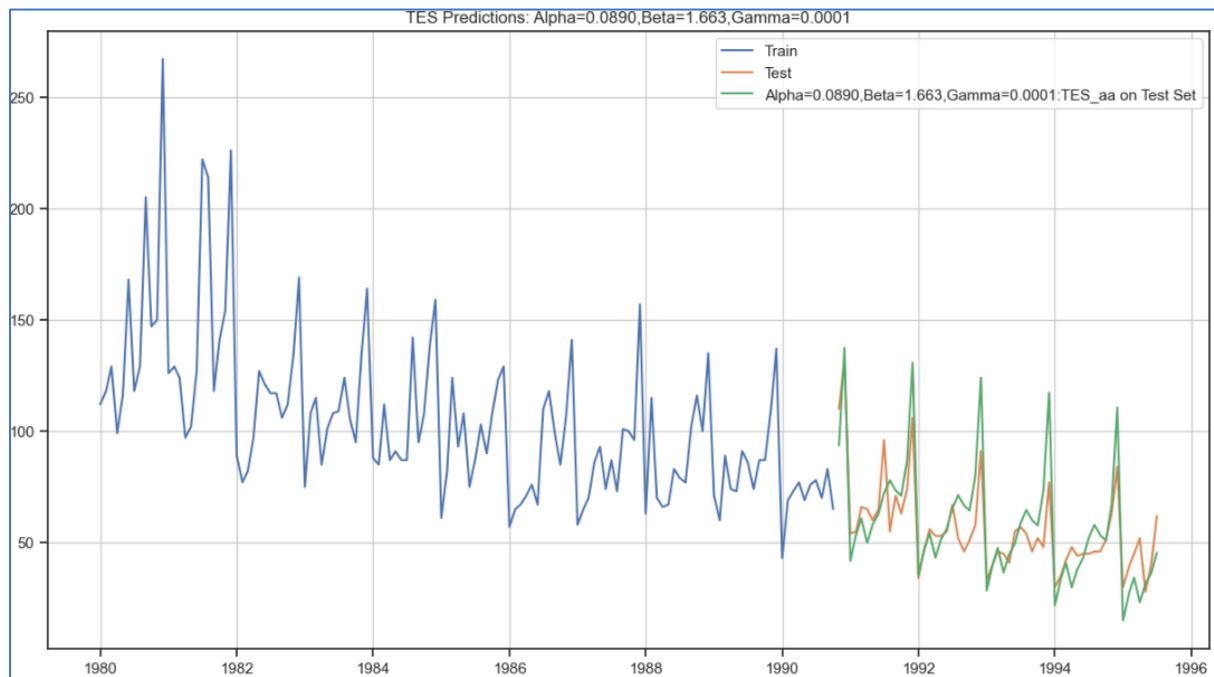


Fig 23. TESaa Model on Test (alpha=0.089 beta=1.663 gamma=0.00016)

8.6.2.2. Model Evaluation

The RMSE of the model is calculated.

```
modelTES_aa= metrics.mean_squared_error(test['Rose'],TES_aa_predict,squared=False)
print("For TESaa forecast on the Test Data, RMSE is %3.3f" %(modelTES_aa))
For TESaa forecast on the Test Data, RMSE is 9.323
```

The RMSE value for the TESaa Model (alpha=0.089 beta=1.663 gamma=0.00016) is 9.323.

OBSERVATIONS:

- The low RMSE value of **9.323** captures the level, trend, and seasonality. But results in a model that has too many parameters fitted to the current data. These parameters have been chosen in such a way that they capture minute details of the training data, including random noise, which do not represent the true underlying patterns of the time series.
- The model with **alpha=0.089** **beta=1.663** and **gamma=0.00016** causes overfitting on the test set.

8.6.2.3. Different values of alpha, beta and gamma (0.01-1)

The different values of alpha, beta and gamma ranging from 0.01-1 that yield lower RMSE scores are obtained.

	Alpha Values	Beta Values	Gamma Values	Test RMSE
4	0.1	0.1	0.5	1.165645e+01
5	0.1	0.1	0.6	1.169280e+01
3	0.1	0.1	0.4	1.170157e+01
116	0.2	0.2	0.7	1.179377e+01
6	0.1	0.1	0.7	1.182788e+01
...	...	...	...	...
979	1.0	0.8	1.0	1.514378e+06
899	0.9	1.0	1.0	2.364386e+06
997	1.0	1.0	0.8	2.441639e+06
998	1.0	1.0	0.9	6.383216e+06
999	1.0	1.0	1.0	1.386836e+07

Table 18. Alpha, Beta, Gamma and RMSE scores for TESaa Model

From the above table, **alpha=0.1**, **beta=0.1** and **gamma=0.5** is found to be a better value with the lowest RMSE score. Hence the TESaa model is forecasted on the test data to see the performance of the model at the lowest RMSE score. The below plot is obtained.

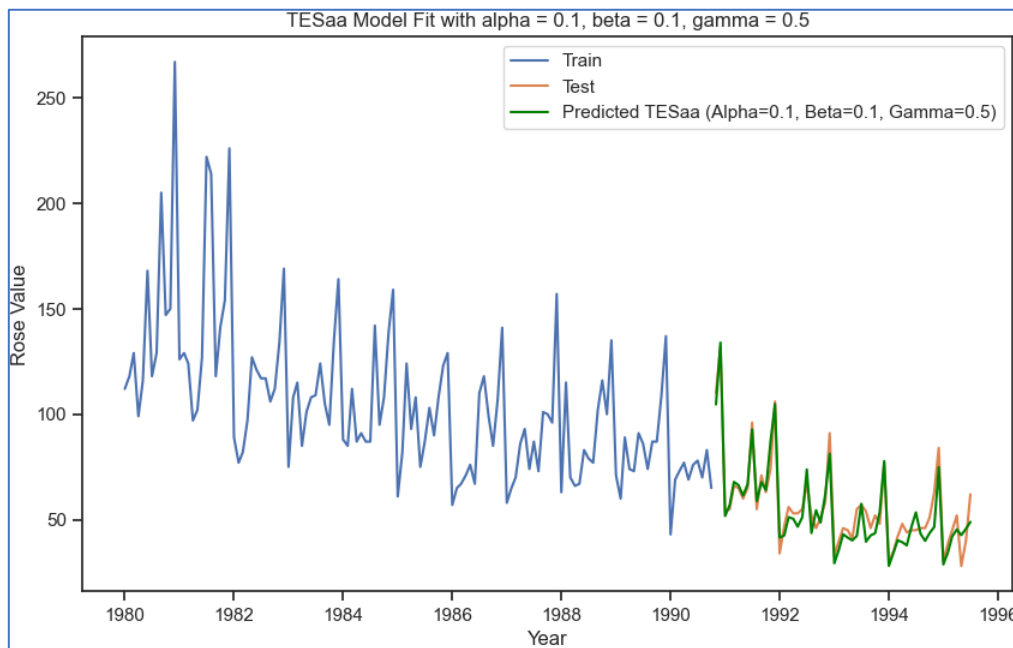


Fig 24. TESaa Model on Test (alpha=0.1 beta=0.1 gamma=0.5)

OBSERVATIONS:

- The TESaa model with **alpha=0.1**, **beta=0.1** and **gamma=0.5** results in a low RMSE of **11.6** which seems to be a good fit.
- The plot further confirms this by showing that the model fits the test data well, indicating that it has learned the underlying patterns in the historical data.

8.6.3. Multiplicative Trend and Multiplicative seasonality

8.6.3.1. Model Building

The TES (multiplicative trend and multiplicative seasonality) model, also called **TESmm Model** is trained on the train set and forecasted on the test set. The below are the parameters.

```
model_TES_mm_autofit.params
{'smoothing_level': 0.0673931131454162,
'smoothing_trend': 5.32103942371749e-09,
'smoothing_seasonal': 1.8748996239692783e-06,
'damping_trend': nan,
'initial_level': 122.9968444058212,
'initial_trend': 0.9937628107193985,
'initial_seasons': array([0.90819386, 1.02851113, 1.12554899, 0.98586501, 1.10729635,
1.19580099, 1.31516967, 1.4050354 , 1.3242819 , 1.29809212,
1.4956679 , 2.06529052]),
'use_boxcox': False,
'lamda': None,
'remove_bias': False}
```

Table 19. TESmm parameters

The TESmm model with **alpha=0.067**, **beta=5.32** and **gamma=1.87** gives the below performance on the test.

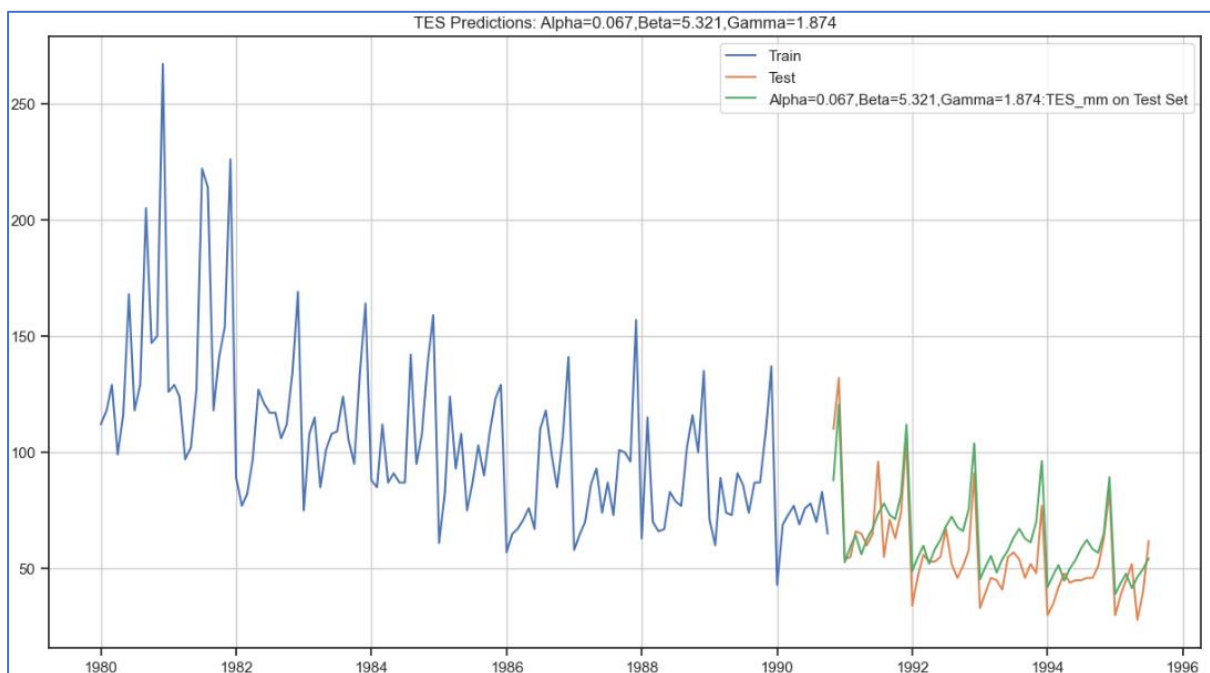


Fig 25. TESmm Model on Test (alpha=0.067 beta=5.32 gamma=1.87)

8.6.3.2. Model Evaluation

The RMSE of the model is calculated.

```
modelTES_mm= metrics.mean_squared_error(test['Rose'],TES_mm_predict,squared=False)
print("For TESmm forecast on the Test Data, RMSE is %3.3f" %(modelTES_mm))
For TESmm forecast on the Test Data, RMSE is 11.716
```

The RMSE value for the TESmm Model (alpha=0.067 beta=5.32 gamma=1.87) is **11.716**.

OBSERVATIONS:

- The **TESmm** model with **alpha=0.067 beta=5.32 gamma=1.87** results in a RMSE of **11.716** which is slightly greater than the TESam model.
- The plot further shows the model fit to a certain level indicating that it has learned the underlying patterns in the historical data to an extent.

8.6.4. Multiplicative Trend and Additive seasonality

8.6.4.1. Model Building

The TES (multiplicative trend and additive seasonality) model, also called **TESma Model** is trained on the train set and forecasted on the test set. The below are the parameters.

```
model_TES_ma_autofit.params
{'smoothing_level': 0.026342174015343802,
'smoothing_trend': 1.4566276787765033e-05,
'smoothing_seasonal': 0.0038378315830715393,
'damping_trend': nan,
'initial_level': 140.96160668626908,
'initial_trend': 0.9937190426186305,
'initial_seasons': array([-20.08515555, -7.44030671,  0.51130607, -9.83542989,
-1.10798068,  4.28962044,  13.40877661,  19.50391666,
 15.68488004,  14.04060508,  30.12332008,  73.0541953 ]),
'use_boxcox': False,
'lamda': None,
'remove_bias': False}
```

Table 20. TESma parameters

The TESma model with **alpha=0.026**, **beta=1.456** and **gamma=0.0038** gives the below performance on the test.

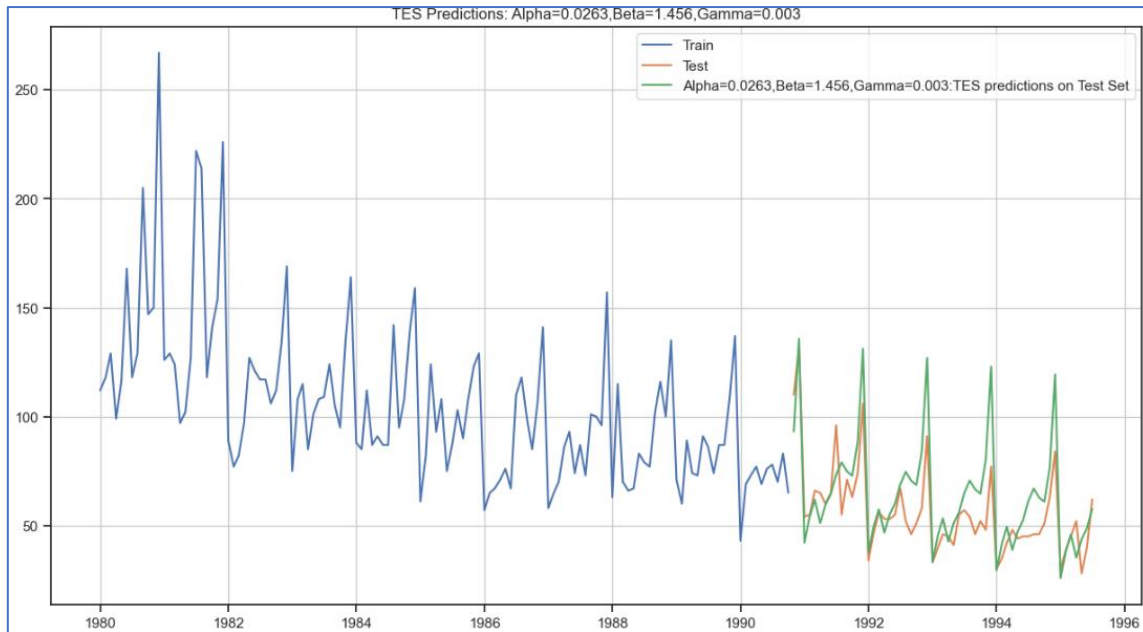


Fig 26. TESma Model on Test (alpha=0.026, beta=1.456 gamma=0.0038)

8.6.4.2. Model Evaluation

The RMSE of the model is calculated.

```
modelTES_ma= metrics.mean_squared_error(test['Rose'],TES_ma_predict,squared=False)
print("For TESma forecast on the Test Data, RMSE is %3.3f" %(modelTES_ma))
For TESma forecast on the Test Data, RMSE is 15.608
```

The RMSE value for the TESma Model (alpha=0.026, beta=1.456 gamma=0.0038) is 15.608.

OBSERVATIONS:

- The TESma model with **alpha=0.026**, **beta=1.456** and **gamma=0.0038** results in a RMSE of **15.608**. But results in a model that has too many parameters fitted to the current data.
- The model with **alpha=0.026**, **beta=1.456** and **gamma=0.0038** causes overfitting on the test set.

8.7. Combined Plots of SES, DES and TES Models

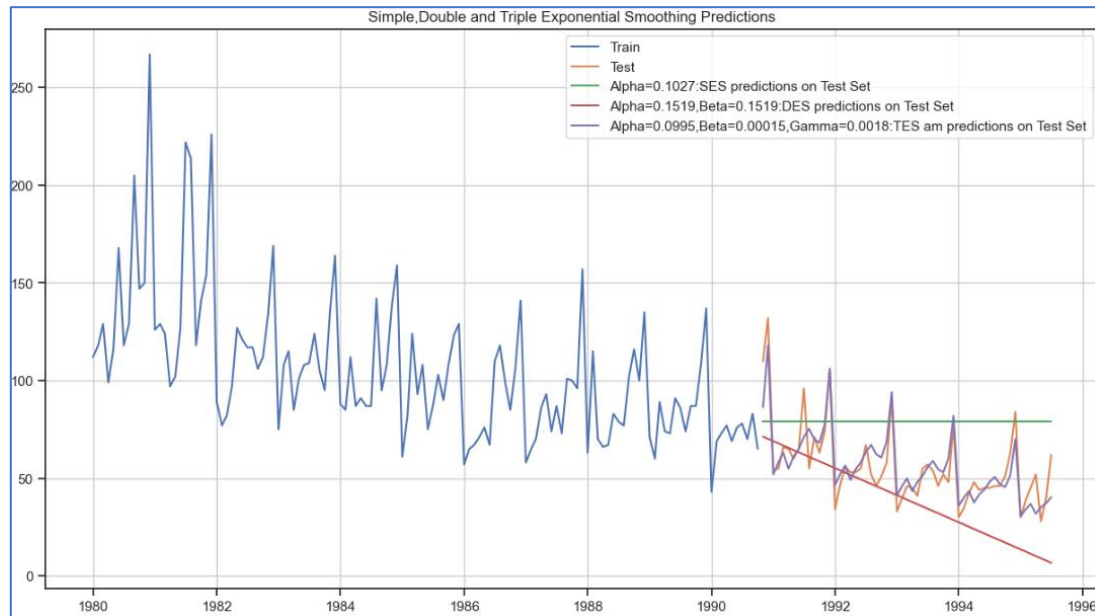


Fig 27. SES, DES and TES Models on Test

OBSERVATIONS:

- The TES Model performs better on the test set compared to SES and DES models.

9. CHECK FOR STATIONARITY

A stationary time series is one whose statistical properties, such as the mean, variance, and autocorrelation, do not change over time. The underlying data should not have trends, seasonal patterns, or other structural changes.

Time series models like **ARIMA** (AutoRegressive Integrated Moving Average) and **SARIMA** (Seasonal AutoRegressive Integrated Moving Average) assume that the series is stationary. Non-stationary data can lead to unreliable parameter estimates and predictions.

The **Augmented Dickey-Fuller (ADF)** test is a statistical test used to check if a time series is **stationary** or not. It specifically tests the below mentioned hypothesis.

Null hypothesis: Time Series is non-stationary.

Alternate hypothesis: Time series is stationary.

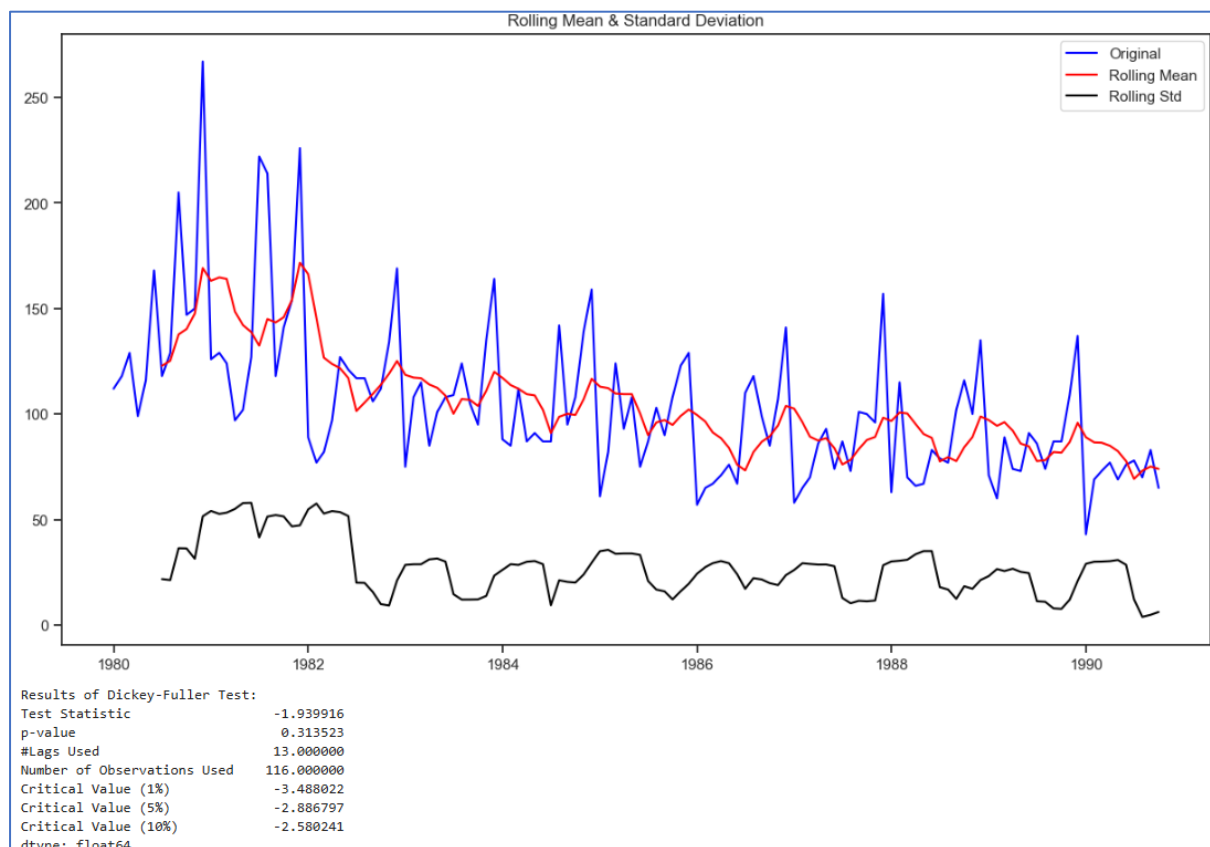


Fig 28. Dicker-Fuller Test-1

The p-value of 0.313 is higher than 0.05, which suggests that we fail to reject the null hypothesis. Therefore, the time series is non-stationary.

Differencing is a technique used to make a time series stationary by removing trends, cycles, or seasonality. It involves subtracting the current value of a time series from the previous value. This method helps eliminate long-term trends or cycles that could otherwise obscure the underlying patterns in the data.

Differencing is done with an order of **d=1**, which gives the below plot making the time series stationary.

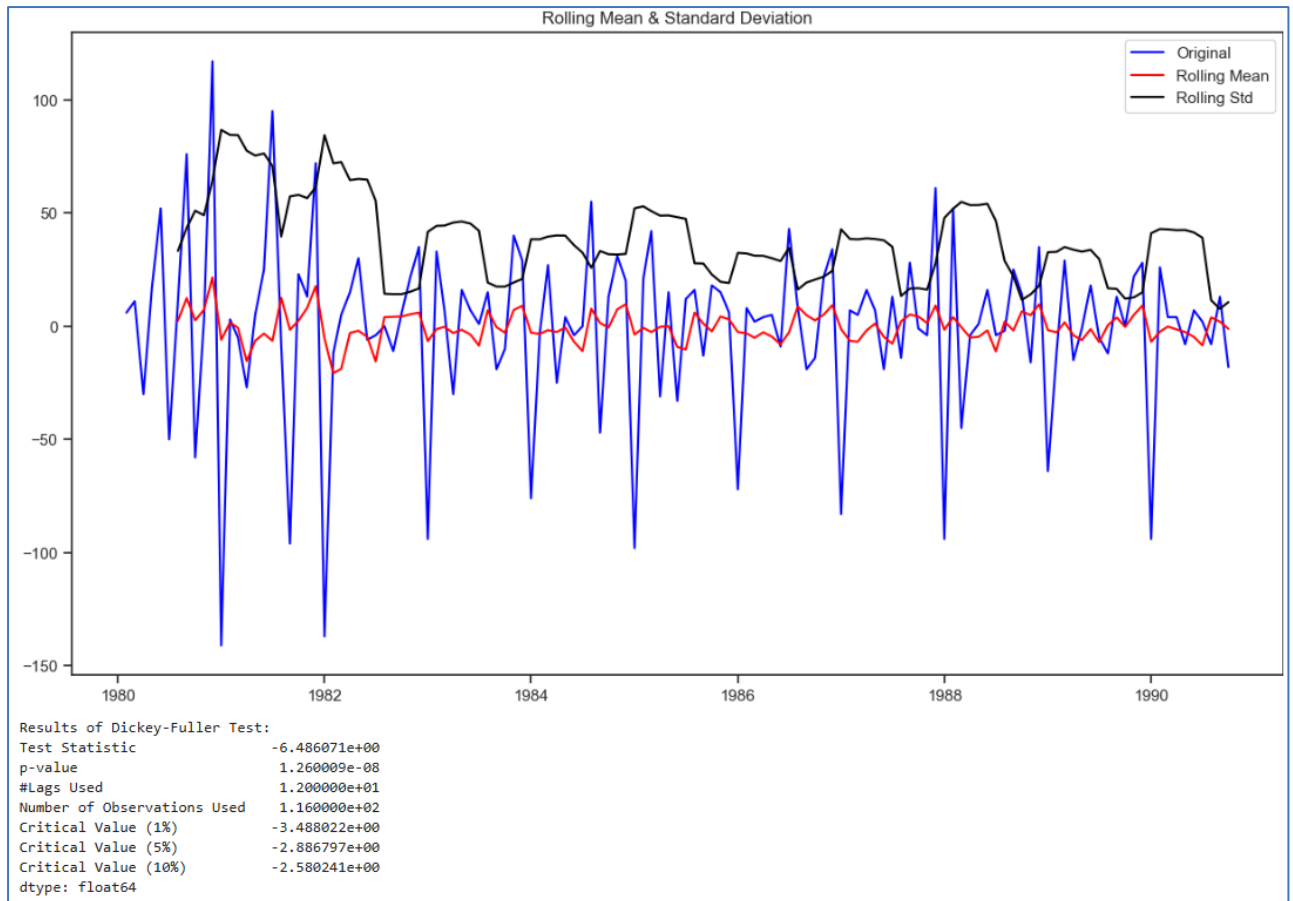


Fig 29. Dicker-Fuller Test-2

OBSERVATIONS:

- The p-value is lesser than 0.05, suggesting we reject the null hypothesis.
- **Time series is stationary.**

10. MODEL BUILDING- STATIONARY DATA

10.1. AutoCorrelation Function (ACF) and Partial AutoCorrelation Function (PACF)

The **stationary time series** is shown below.

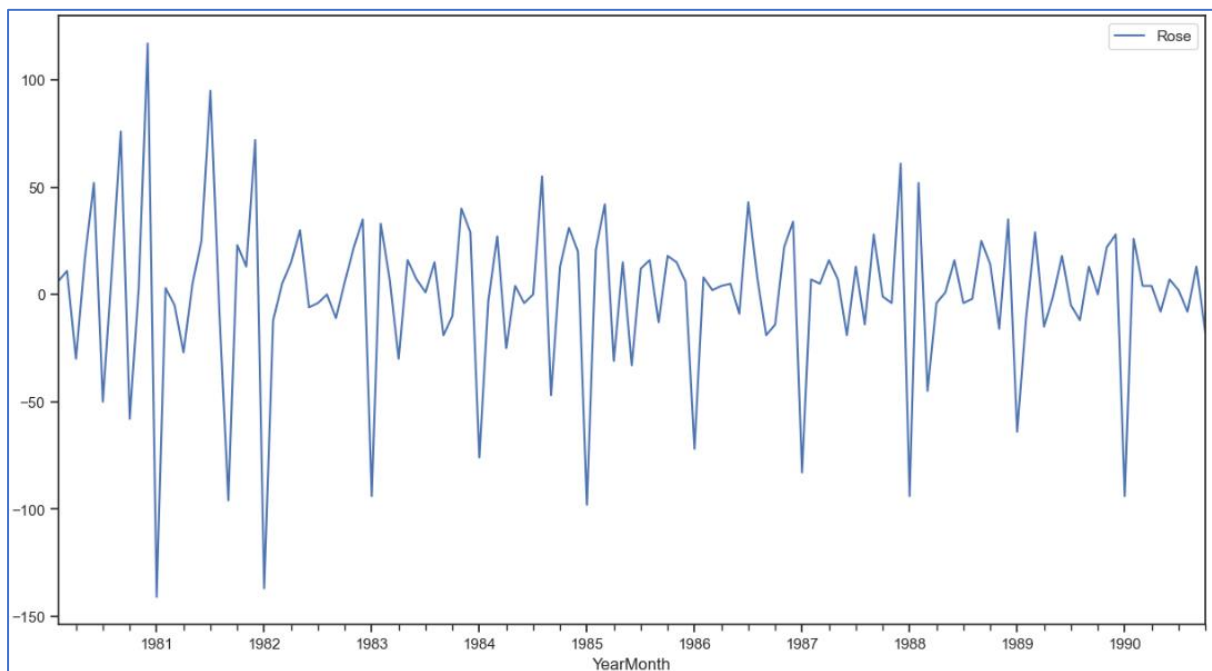


Fig 30. Stationary time series

The **ACF** measures the correlation of a time series with its own lagged values. It essentially tells you how current values in a time series are correlated with past values at different time lags.

The **PACF** measures the correlation between the time series and its lagged values after removing the effect of the intervening lags.

- If the ACF decreases gradually (exponentially), it indicates an **AR process** (AutoRegressive process).
- If the PACF cuts off after a few lags, this suggests an **AR process** with the order equal to the number of lags before the cut-off.
- If the PACF decays gradually, it suggests a **MA process** (Moving Average).

The ACF and PACF plots are obtained. Both ACF and PACF plots help in determining the parameters for **ARIMA** models, such as **p, d and q**.

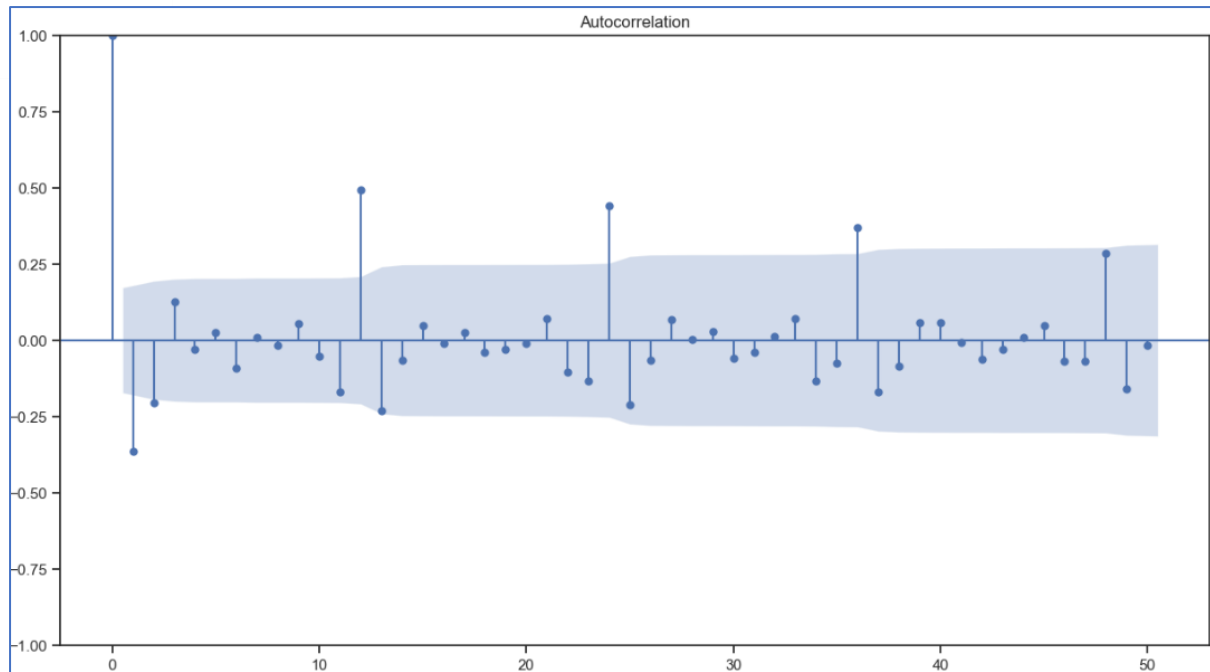


Fig 31. ACF Plot

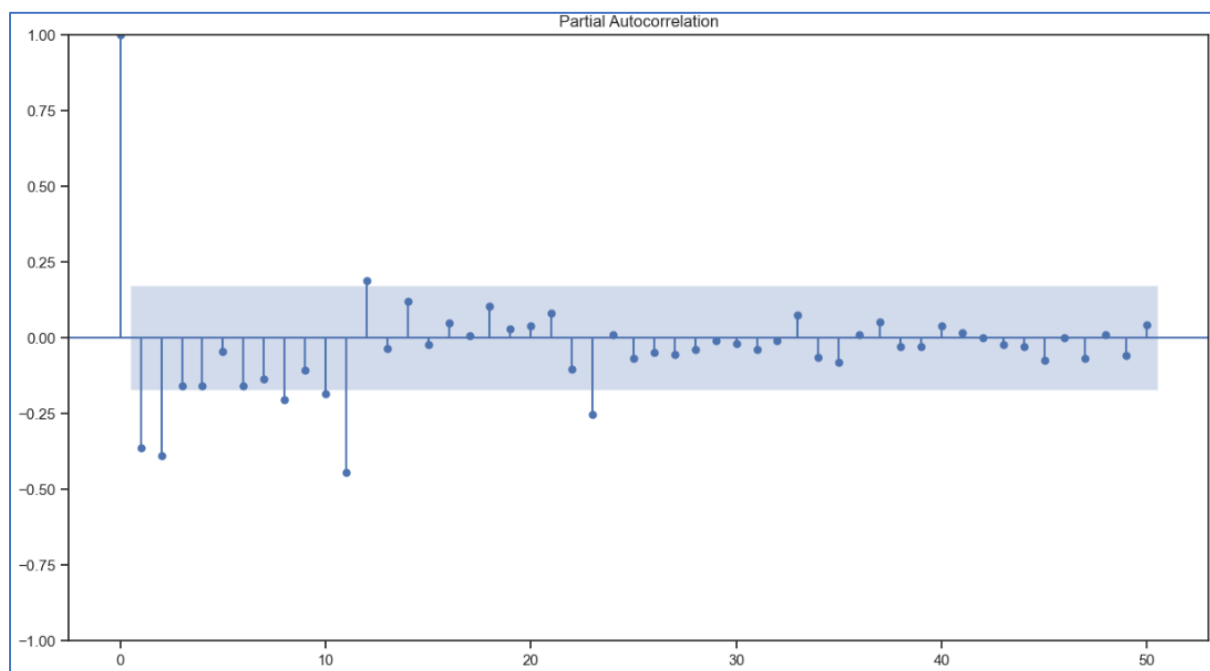


Fig 32. PACF Plot

OBSERVATIONS:

- The ACF decreases gradually which it indicates an AR process.
- The AR value obtained from the PACF plot is **p=2**. The value at lag 2 cuts the threshold area.
- The MA value obtained from ACF plot is **q=1**. The value at lag 1 cuts the threshold area significantly.

10.2. Manual ARIMA

The ARIMA (AutoRegressive Integrated Moving Average) model is a popular statistical model used for time series forecasting. It combines three key components:

- AutoRegressive (AR): A model that uses the relationship between an observation and a specified number of lagged observations (previous time steps).
- Integrated (I): The differencing of raw observations to make the time series stationary.
- Moving Average (MA): A model that uses the dependency between an observation and a residual error from a moving average model applied to lagged observations.

Manually fitting an ARIMA (AutoRegressive Integrated Moving Average) model involves understanding the individual components and how they interact to make a time series model.

10.2.1. Model Building

From the ACF and PACF plots, the AR and MA values have been found to be **p=2** and **q=1**. The time series is made stationary by differencing by **d=1**. Hence the parameters for manual ARIMA is **order=(2,1,1)**.

The below results are obtained by fitting the train set with the manual ARIMA model.

SARIMAX Results						
=====						
Dep. Variable:	Rose		No. Observations:	130		
Model:	ARIMA(2, 1, 1)		Log Likelihood	-626.507		
Date:	Sat, 07 Dec 2024		AIC	1261.014		
Time:	22:19:52		BIC	1272.453		
Sample:	01-01-1980		HQIC	1265.662		
	- 10-01-1990					
Covariance Type:	opg					
=====						
	coef	std err	z	P> z	[0.025	0.975]

ar.L1	0.1723	0.079	2.168	0.030	0.017	0.328
ar.L2	-0.0938	0.105	-0.893	0.372	-0.300	0.112
ma.L1	-0.9029	0.061	-14.824	0.000	-1.022	-0.783
sigma2	956.4602	87.026	10.990	0.000	785.892	1127.029
=====						
Ljung-Box (L1) (Q):			0.01	Jarque-Bera (JB):		46.90
Prob(Q):			0.94	Prob(JB):		0.00
Heteroskedasticity (H):			0.32	Skew:		0.92
Prob(H) (two-sided):			0.00	Kurtosis:		5.30
=====						
Warnings:						
[1] Covariance matrix calculated using the outer product of gradients (complex-step).						

Table 21. Manual ARIMA results

The manual ARIMA model is forecasted on the test set to obtain the below plot.

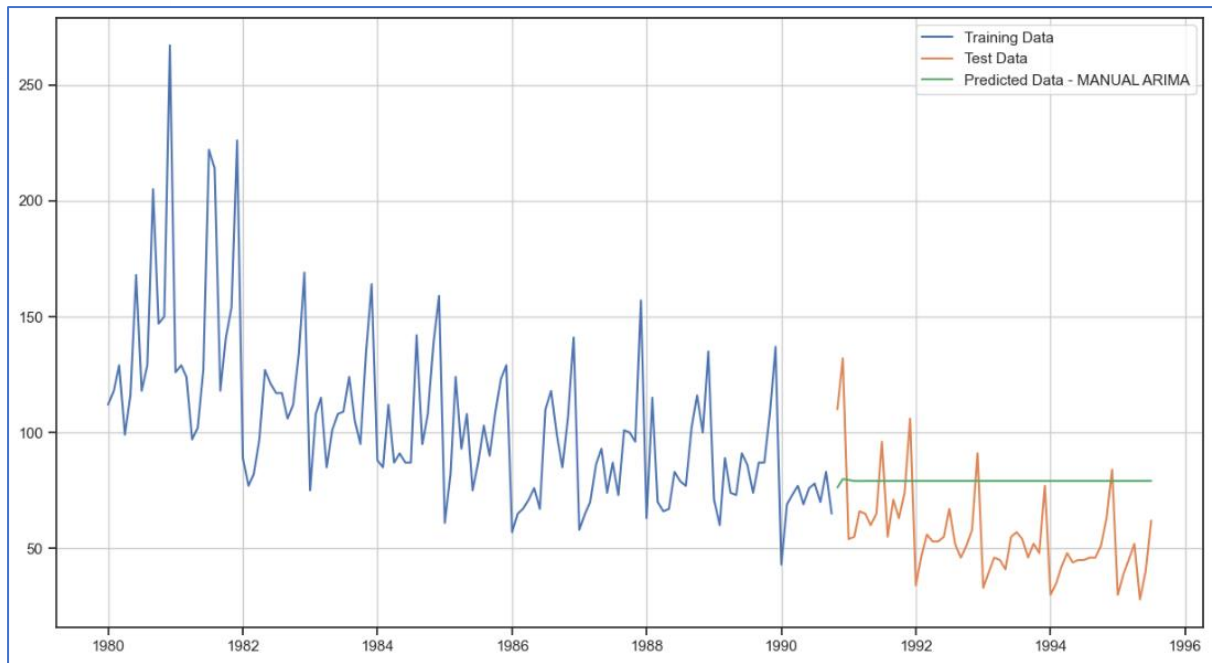


Fig 33. Manual ARIMA

10.2.2. Model Evaluation

The RMSE of the model is calculated.

```
from sklearn.metrics import mean_squared_error
rmse = mean_squared_error(test['Rose'], predicted_ARIMA, squared=False)
print(rmse)

30.358977692992305
```

The RMSE value for the Manual ARIMA model at order=(2,1,1) is 30.35.

OBSERVATIONS:

- From the manual ARIMA results, it is seen that the **AR lag 1** and **MA lag 1** values are significant for forecasting the time series.
- **RMSE value of 30.35** indicates that the predictions made by the ARIMA model with the **order=(2, 1, 1)** are not very close to the actual values. In other words, the model is making substantial errors in its predictions.
- The model is not effectively capturing the underlying patterns of the time series data, as suggested by the relatively high RMSE score. The lower the RMSE value, the better the model's predictions.

10.3. Auto ARIMA

Auto ARIMA is a method that automates the process of selecting the best ARIMA model by evaluating a range of possible models and selecting the one with the **lowest Akaike Information Criterion (AIC) value**. Lower AIC values indicate better models, as they strike a good balance between fit and model complexity.

10.3.1. Model Building

The different combinations of **p,d,q** values that yield lower RMSE scores are obtained. The below table shows the AIC values.

	param	AIC
11	(2, 1, 3)	1258.119650
15	(3, 1, 3)	1258.436640
2	(0, 1, 2)	1259.247780
6	(1, 1, 2)	1259.473205
5	(1, 1, 1)	1260.036763
3	(0, 1, 3)	1260.132819
9	(2, 1, 1)	1261.014076
1	(0, 1, 1)	1261.327444
10	(2, 1, 2)	1261.472001
7	(1, 1, 3)	1261.472191
13	(3, 1, 1)	1261.969098
14	(3, 1, 2)	1263.331767
12	(3, 1, 0)	1276.842717
8	(2, 1, 0)	1278.135281
4	(1, 1, 0)	1297.077294
0	(0, 1, 0)	1313.175861

Table 22. Auto ARIMA AIC and RMSE values

Using the above table, the low AIC value gives the **order=(2,1,3)**. The auto ARIMA model is fitted on the train data using the p,d,q values obtained by lowest AIC order.

SARIMAX Results

Dep. Variable:

Rose

No. Observations:

130

Model:

ARIMA(2, 1, 3)

Log Likelihood

-623.060

Date:

Sat, 07 Dec 2024

AIC

1258.120

Time:

22:19:57

BIC

1275.279

Sample:

01-01-1980

HQIC

1265.092

- 10-01-1990

Covariance Type:

opg

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	-1.6991	0.059	-28.742	0.000	-1.815	-1.583
ar.L2	-0.8618	0.058	-14.889	0.000	-0.975	-0.748
ma.L1	0.9634	0.101	9.554	0.000	0.766	1.161
ma.L2	-0.6769	0.106	-6.364	0.000	-0.885	-0.468
ma.L3	-0.8826	0.091	-9.718	0.000	-1.061	-0.705
sigma2	887.2555	96.548	9.190	0.000	698.024	1076.487

Ljung-Box (L1) (Q):

0.79

Jarque-Bera (JB):

35.17

Prob(Q):

0.38

Prob(JB):

0.00

Heteroskedasticity (H):

0.31

Skew:

0.81

Prob(H) (two-sided):

0.00

Kurtosis:

4.97

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

Table 23. Auto ARIMA results

The auto ARIMA model is forecasted on the test set to obtain the below plot.

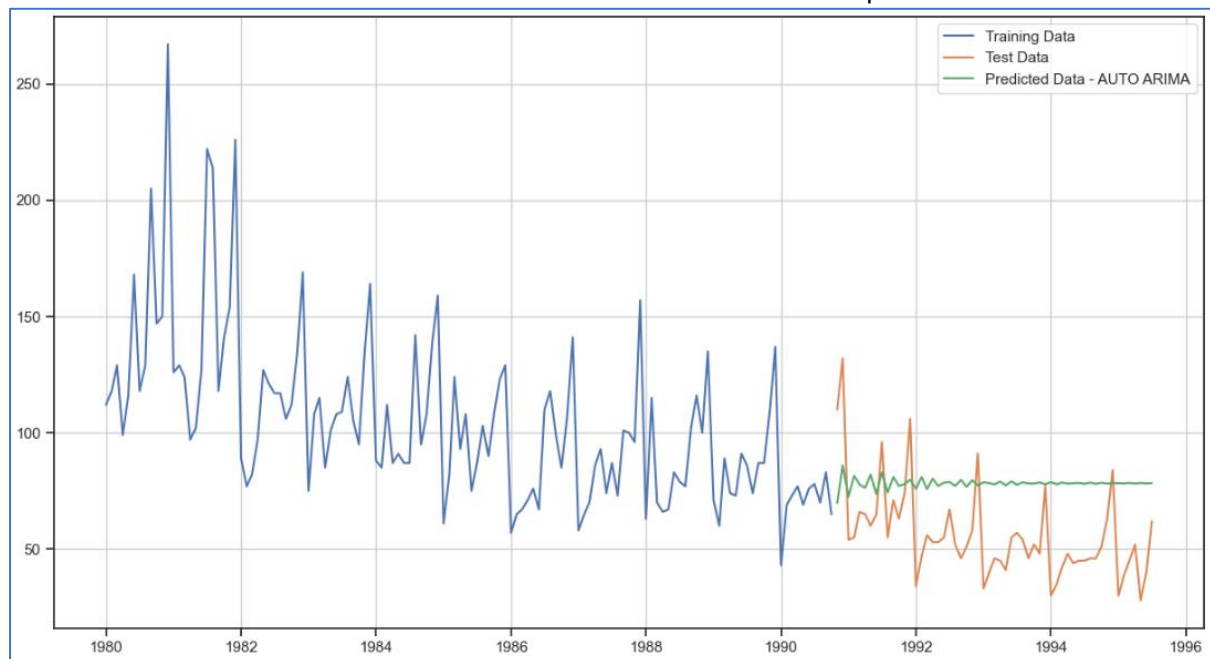


Fig 34. Auto ARIMA

10.3.2. Model Evaluation

The RMSE of the model is calculated.

```
from sklearn.metrics import mean_squared_error
rmse = mean_squared_error(test['Rose'],predicted_auto_ARIMA,squared=False)
print(rmse)
29.55734778229556
```

The RMSE value for the Manual ARIMA model at order=(2,1,3) is 29.557

OBSERVATIONS:

- From the auto ARIMA results, it is seen that the **AR lag 1 and lag 2** and **MA lag 1, lag 2 and lag 3** values are significant for forecasting the time series.
- **RMSE value of 29.557** indicates that the predictions made by the ARIMA model with the **order=(2, 1, 3)** are not very close to the actual values.
- The model is not effectively capturing the underlying patterns of the time series data, as suggested by the relatively high RMSE score.

10.4. Manual SARIMA

SARIMA (Seasonal ARIMA), also known as Seasonal Autoregressive Integrated Moving Average, is an extension of the ARIMA model that explicitly accounts for seasonality in time series data.

The SARIMA model is often denoted as **SARIMA (p,d,q) (P,D,Q,m)**, where the (p,d,q) parameters are the same as ARIMA and additionally have (P,D,Q,m) which are the seasonal parameters and m denotes the number of periods in a season.

10.4.1. Model Building

From the ACF and PACF plots, the AR and MA values have been found to be **p=2** and **q=1**. The time series is made stationary by differencing by **d=1**. Hence the parameters for manual SARIMA is **order=(2,1,1)** and **seasonal order=(2,0,1,12)**.

The below results are obtained by fitting the train set with the manual SARIMA model.

SARIMAX Results						
=====						
Dep. Variable:	Rose		No. Observations:	130		
Model:	SARIMAX(2, 1, 1)x(2, 0, 1, 12)		Log Likelihood	-433.604		
Date:	Sat, 07 Dec 2024		AIC	881.207		
Time:	22:20:04		BIC	899.650		
Sample:	01-01-1980		HQIC	888.677		
	- 10-01-1990					
Covariance Type:	opg					
=====						
	coef	std err	z	P> z	[0.025	0.975]

ar.L1	0.1750	0.122	1.440	0.150	-0.063	0.413
ar.L2	-0.1142	0.091	-1.260	0.208	-0.292	0.063
ma.L1	-1.0000	419.570	-0.002	0.998	-823.342	821.342
ar.S.L12	0.3209	0.096	3.336	0.001	0.132	0.509
ar.S.L24	0.2907	0.075	3.854	0.000	0.143	0.439
ma.S.L12	0.1055	0.151	0.699	0.484	-0.190	0.401
sigma2	254.2794	1.07e+05	0.002	0.998	-2.09e+05	2.09e+05
=====						
Ljung-Box (L1) (Q):	0.05		Jarque-Bera (JB):	3.96		
Prob(Q):	0.82		Prob(JB):	0.14		
Heteroskedasticity (H):	0.93		Skew:	0.45		
Prob(H) (two-sided):	0.82		Kurtosis:	3.33		
=====						
Warnings:						
[1] Covariance matrix calculated using the outer product of gradients (complex-step).						

Table 24. Manual SARIMA results

The manual SARIMA model is forecasted on the test set to obtain the below plot.

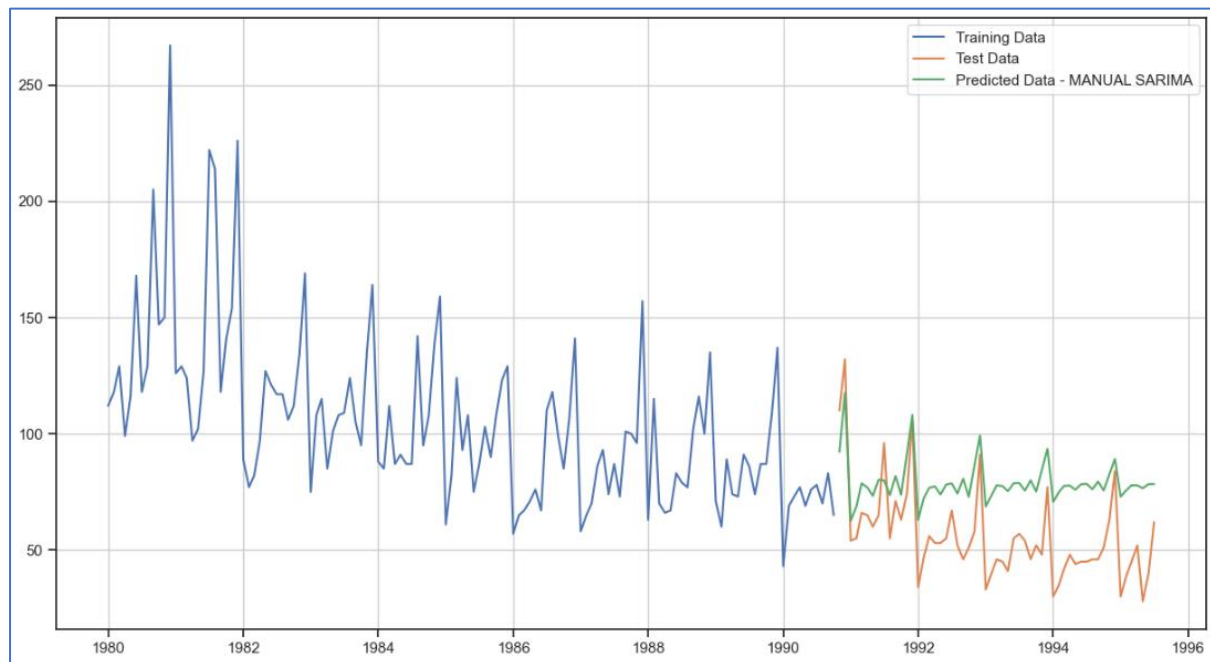


Fig 35. Manual SARIMA

10.4.2. Model Evaluation

The RMSE of the model is calculated.

```
from sklearn.metrics import mean_squared_error
rmse = mean_squared_error(test['Rose'], predicted_SARIMA, squared=False)
print(rmse)
26.648182892994974
```

The RMSE value for the Manual SARIMA model at order=(2,1,1)(2,0,1,12) is 26.64.

OBSERVATIONS:

- From the manual SARIMA results, it is seen that the **seasonal AR lags 12 and 24** values are significant for forecasting the time series.
- The RMSE value of **26.64** fails to capture the true underlying patterns of the time series.
- The model with **order=(2,1,1)(2,0,1,12)** is not a good fit on the test set.

10.5. Auto SARIMA

Auto SARIMA is an automatic method to select the best SARIMA model for a given time series data, by evaluating different combinations of seasonal and non-seasonal ARIMA parameters and selecting the one with the **lowest Akaike Information Criterion (AIC)**.

10.5.1. Model Building

The different combinations of **(p,d,q)(P,D,Q,12)** values that yield lower RMSE scores are obtained. The below table shows the AIC values.

	param	seasonal	AIC
26	(0, 1, 2)	(2, 0, 2, 12)	871.075238
53	(1, 1, 2)	(2, 0, 2, 12)	873.003875
80	(2, 1, 2)	(2, 0, 2, 12)	874.213961
69	(2, 1, 1)	(2, 0, 0, 12)	879.792363
78	(2, 1, 2)	(2, 0, 0, 12)	880.763857

Table 25. Auto SARIMA AIC and RMSE values

Using the above table, the low AIC value gives the **order=(0,1,2)(2,0,2,12)**. The auto SARIMA model is fitted on the train data using the (p,d,q)(P,D,Q,m) values obtained by lowest AIC order.

SARIMAX Results						
=====						
Dep. Variable:	y	No. Observations:	130			
Model:	SARIMAX(0, 1, 2)x(2, 0, 2, 12)	Log Likelihood	-428.538			
Date:	Sat, 07 Dec 2024	AIC	871.075			
Time:	22:21:06	BIC	889.450			
Sample:	0	HQIC	878.516			
	- 130					
Covariance Type:	opg					
=====						
	coef	std err	z	P> z	[0.025	0.975]

ma.L1	-0.8367	239.199	-0.003	0.997	-469.658	467.985
ma.L2	-0.1633	39.042	-0.004	0.997	-76.684	76.357
ar.S.L12	0.3494	0.079	4.408	0.000	0.194	0.505
ar.S.L24	0.3067	0.075	4.103	0.000	0.160	0.453
ma.S.L12	0.0454	0.134	0.338	0.735	-0.218	0.309
ma.S.L24	-0.0912	0.145	-0.628	0.530	-0.376	0.193
sigma2	250.7786	6e+04	0.004	0.997	-1.17e+05	1.18e+05
=====						
Ljung-Box (L1) (Q):	0.09	Jarque-Bera (JB):	3.10			
Prob(Q):	0.76	Prob(JB):	0.21			
Heteroskedasticity (H):	0.88	Skew:	0.43			
Prob(H) (two-sided):	0.71	Kurtosis:	3.05			
=====						
Warnings:						
[1] Covariance matrix calculated using the outer product of gradients (complex-step).						

Table 26. Auto SARIMA results

The auto SARIMA model is forecasted on the test set to obtain the below plot.

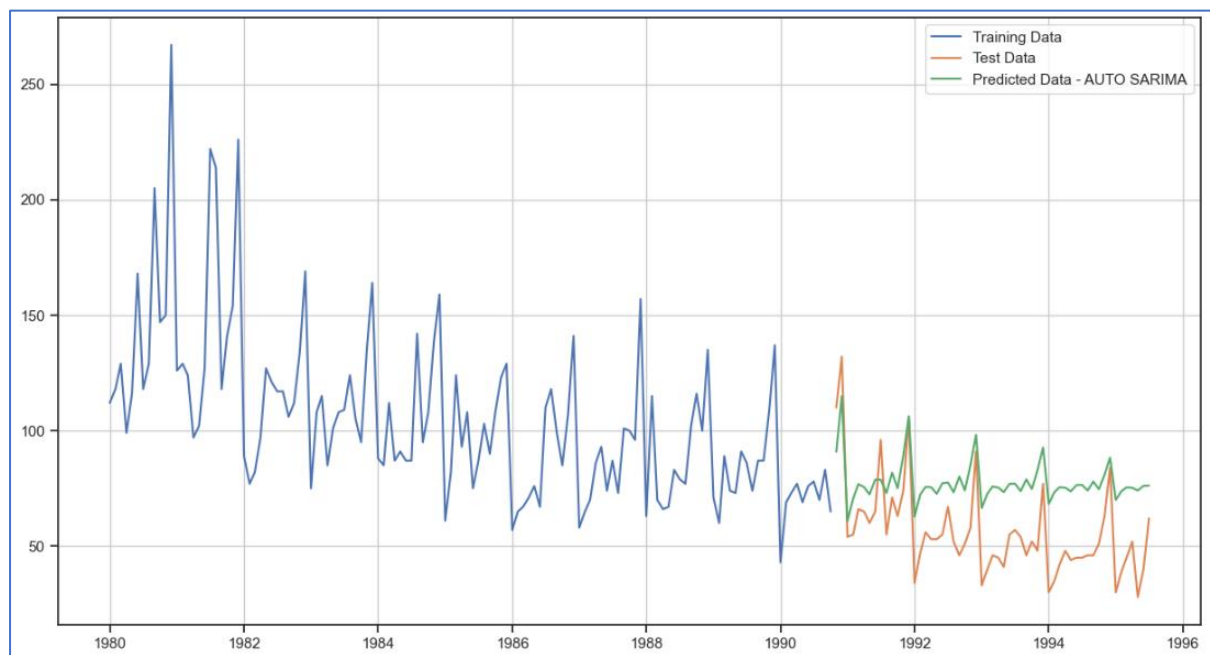


Fig 36. Auto SARIMA

10.5.2. Model Evaluation

The RMSE of the model is calculated.

```
rmse = mean_squared_error(test['Rose'],predicted_auto_SARIMA.predicted_mean,squared=False)
print(rmse)
25.344039370629236
```

The RMSE value for the Auto SARIMA model at order=(0,1,2)(2,0,2,12) is 25.344.

A **SARIMA diagnostics plot** is used to assess the quality of a fitted SARIMA (Seasonal Autoregressive Integrated Moving Average) model. The goal is to check the residuals (errors) of the model to ensure they resemble white noise, meaning that they should have a mean of zero, no autocorrelation, and constant variance.

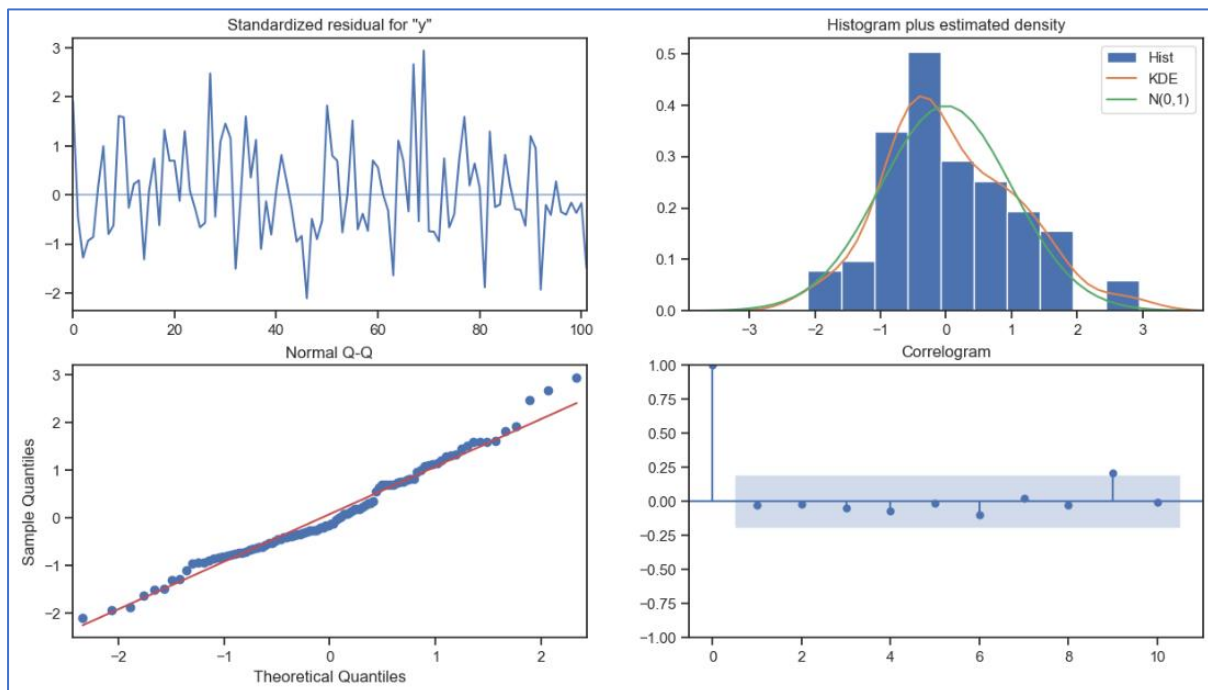


Fig 37. Auto SARIMA Diagnostics Plot

OBSERVATIONS:

- The residual plot does not show any pattern.
- The histogram plot shows the distribution of the residuals. There are differences between the orange and green line, which indicate that the model is not a very good fit.
- The Q-Q plot shows a line lying along the red line, except for some values at the end, which is exactly happening in this case. The distribution of the residuals is close to normal.
- The correlogram plot is the ACF plot of the residuals in which 95% of the correlations for lag >0 is not significant (within the blue shades). The model has captured the data to some level.

10.6. Comparison of ARIMA and SARIMA models

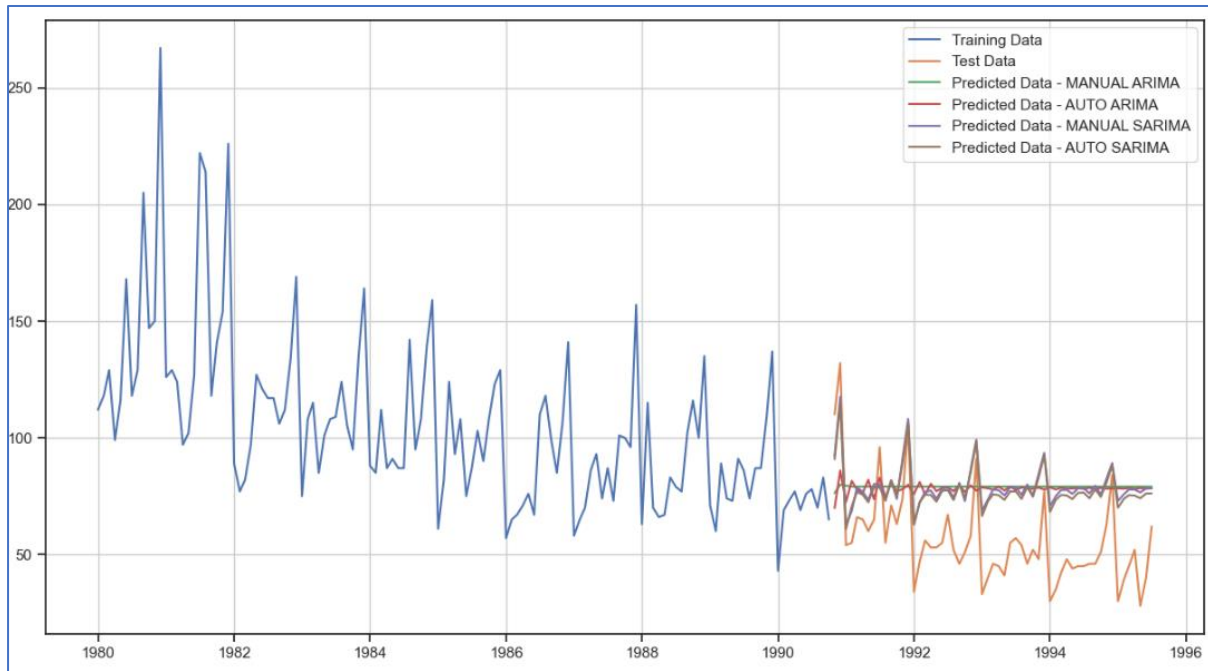


Fig 38. Comparison of ARIMA and SARIMA models

OBSERVATIONS:

- The SARIMA models perform better than the ARIMA models, but however the SARIMA models fail to capture the patterns of underlying historical data making it unfit for the rose wine time series data.

11. COMPARISON OF PERFORMANCE OF THE MODELS

The RMSE values of all the models developed so far are compared and sorted in ascending order to identify the model with the lowest errors, thereby assessing its performance.

	Test RMSE
Model_6: TES (add trend + mul season), Alpha=0.1, Beta=1.0 Gamma=0.2	9.130274
Model_6: TES (add trend + mul season), Alpha=0.0995, Beta=0.00015, Gamma=0.0018	9.322909
Model_6: TES (add trend + add season): Alpha=0.1, Beta=0.1, Gamma=0.5:TES	11.656452
Model_6: TES (mul trend + mul season), Alpha=0.067, Beta=5.321, Gamma=1.874	11.716085
Model_3: 2pointTrailingMovingAverage	11.801167
Model_6: TES (add trend + add season): Alpha=0.0890, Beta=1.663, Gamma=0.0001	13.962776
Model_3: 4pointTrailingMovingAverage	15.366538
Model_6: TES (mul trend + add season), Alpha=0.076, Beta=0.021, Gamma=0.475	15.608004
Model_3: 6pointTrailingMovingAverage	15.863483
Model_3: 9pointTrailingMovingAverage	16.342156
Model_5: DES, Alpha=0.20, Beta=0.18	17.287718
Model_1: RegressionOnTime	17.356199
Model_4: SES, Alpha=0.99	21.839882
AUTO SARIMA(0,1,2)(2,0,2,12)	25.344039
Model_5: DES, Alpha=0.1519, Beta=0.1519	26.049952
MANUAL SARIMA(2,1,1) (2,0,1,12)	26.648183
AUTO ARIMA(2,1,3)	29.557348
Model_4: SES, Alpha=0.1027	30.188455
MANUAL ARIMA(2,1,1)	30.358978
Model_2: SimpleAverageModel	52.412167

Table 27. Comparison of RMSE values of Models

OBSERVATIONS:

- Overall, the **Triple Exponential Smoothing (TES)** models exhibit a lower range of **RMSE** values, ranging from **9.1 to 11.7**.
- The **Moving Average** model at **window 2** also performs well with a RMSE of **11.8**.
- Among the TES models, the **TES model with additive trend and multiplicative seasonality (TESam)**, with parameters **alpha=0.1, beta=0.1, and gamma=0.2**, outperforms the others, achieving the **lowest RMSE score of 9.1**.
- Therefore, the **TESam model (alpha=0.1 beta=0.1 gamma=0.2)** is the best performing model for the test set of the time series.

12. REBUILDING THE BEST MODEL USING THE ENTIRE DATA

12.1. Model Building

The **TESam model (alpha=0.1 beta=0.1 gamma=0.2)** is the best performing model for the test set of the time series. This model is rebuilt on the entire data to test the performance on the entire time series data.

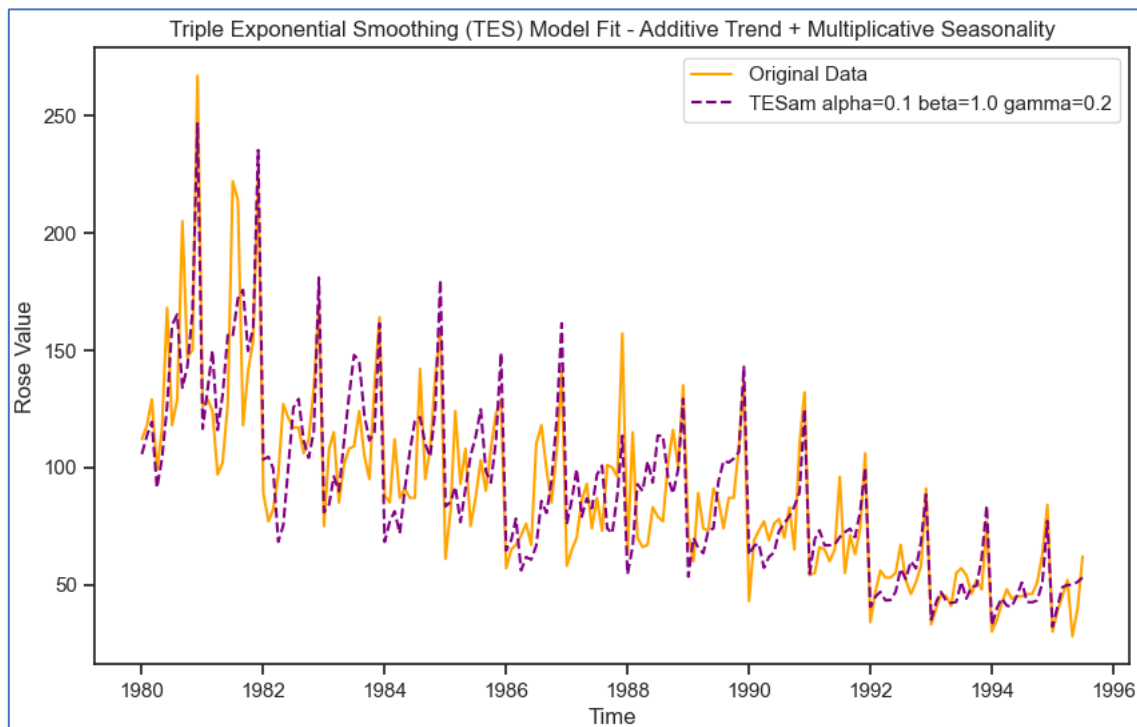


Fig 39. TESam model (alpha=0.1 beta=0.1 gamma=0.2) on entire data

12.2. Model Evaluation

The RMSE of the model is calculated.

```
rmse = np.sqrt(mean_squared_error(df1['Rose'], fitted_values))
print(f'Root Mean Squared Error (RMSE) on the entire dataset: {rmse}')
Root Mean Squared Error (RMSE) on the entire dataset: 19.479023241749918
```

The RMSE value for the TESam model (alpha=0.1 beta=1.0 gamma=0.2) is 19.47.

OBSERVATIONS:

- The **TESam model (with alpha=0.1, beta=1.0, and gamma=0.2)** applied to the entire dataset shows a lower RMSE value, and the plots indicate a good fit across the entire data.
- This suggests that the **TESam model** could be the most suitable model for forecasting rose wine sales.

13. FORECAST FOR THE NEXT 12 MONTHS

The **TESam model (with $\alpha=0.1$, $\beta=1.0$, and $\gamma=0.2$)** is used for obtaining the forecast for the next 12 months.

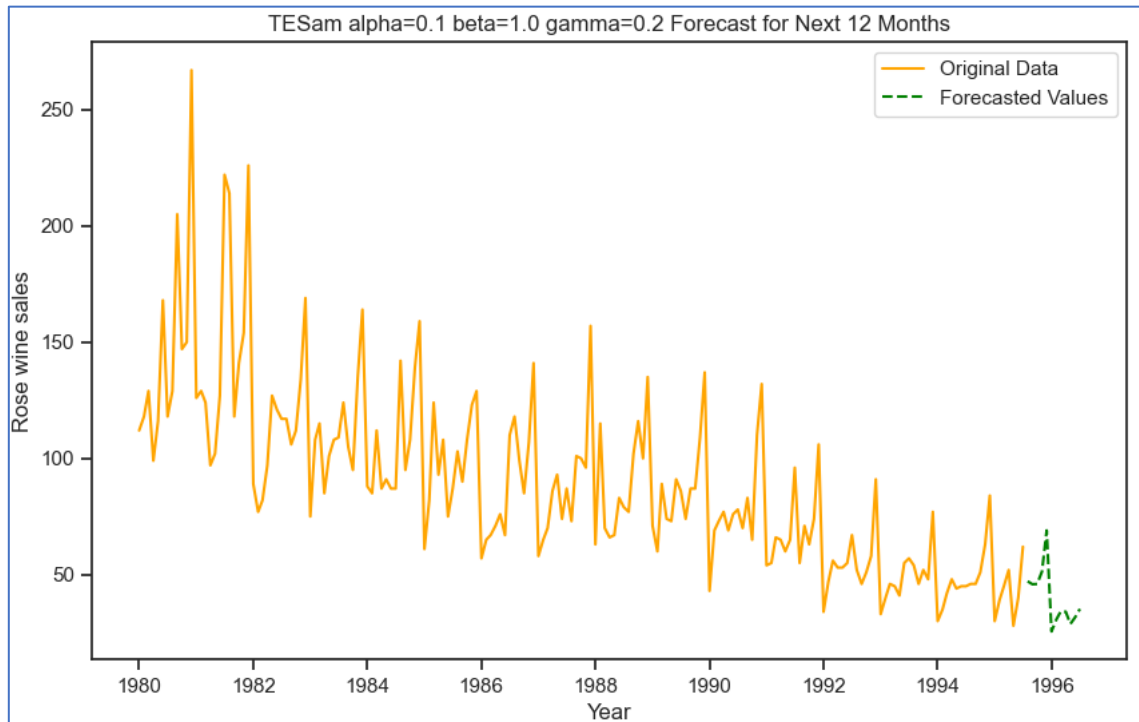


Fig 40. Forecast for next 12 months

OBSERVATIONS:

- The model predicts that the sales of rose wine in 1996 will follow a similar seasonal pattern to the previous years.
- It is expected that the sales in 1996 will be slightly lower than those of 1995, indicating a decreasing trend.
- Specifically, the forecast suggests that sales will peak at approximately **80 units in December**, reflecting a significant increase during the holiday season.
- On the other hand, sales are expected to dip as low as below **25 units** during the off-peak months, highlighting the typical seasonal fluctuation.
- Overall, the model forecasts a **slightly lower overall sales volume** for 1996 with notable seasonal variations.

14. ACTIONABLE INSIGHTS AND RECOMMENDATIONS

To increase rose wine sales based on the forecast provided by the model, several strategies can be implemented, taking into account the seasonal trends and growth forecast for 1996.

- **Maximize Holiday Season Sales:** Since the forecast indicates a peak in December with sales reaching 275 units, targeted promotional campaigns during the holiday season can be implemented. This could include special discounts or limited-time offers to increase demand during this period.
- **Partnerships with Event Organizers:** Partnering with event organizers, catering services or restaurants can promote the sales of rose wine during holiday events.
- **Introduce Seasonal Promotions:** During the off-peak months when sales dip below 25 units, creating off-season promotions or discounts to maintain steady sales throughout the year by considering bulk purchase discounts or loyalty programs to keep customers engaged.
- **Seasonal Flavors:** Introducing special seasonal flavors or limited-edition bottles to make the product more appealing during off-peak months can generate interest and attract customers to buy during typically low-sales months.
- **Stock Management:** Based on the forecast, ensuring that the inventory levels are aligned with the predicted seasonal variations by stocking more units ahead of the holiday season to meet higher demand and reducing stock during off-peak months could help.
- **Expand Distribution Channels:** Collaborating with more retailers, both online and offline, expands the availability of rose wine, especially during peak seasons.
- **Global Expansion:** If feasible, exploring international markets where rose wine is in demand, particularly in regions with growing middle-class consumers or places where holiday celebrations heavily feature wine would drive the sales.

By utilizing the insights from the sales forecast model, the ABC Estate Wines can strategically focus their efforts on maximizing peak season sales, while also maintaining engagement and encouraging purchases during the off-peak months.